

Deep Learning - lecture 1

An introduction to deep learning, models and performance metrics

Prof. Stefano Carrazza

University of Milan and INFN Milan

Introduction

Lectures organization

- **Lecturer:** Stefano Carrazza
- **E-mail:** stefano.carrazza@unimi.it
- **Office:** Dipartimento di Fisica, Via Celoria 16, room DC/1/4
- **Ariel:** <https://scarrazzadla.ariel.ctu.unimi.it/> (announcements & links)

Lectures

- **42 hours**
- **Exam:** Development of a project (written report + code + talk).
The project topic has to be previously discussed with the lecturer.

1

0/6 min

Why lectures on deep learning?

2

Why lectures on deep learning?

because

- it is an essential set of **algorithms** for **building models** in science,

2

Why lectures on deep learning?

because

- it is an essential set of **algorithms** for **building models** in science,
- fast development of new **tools and algorithms** in the past years,

2

Why lectures on deep learning?

because

- it is an essential set of **algorithms** for **building models** in science,
- fast development of new **tools and algorithms** in the past years,
- nowadays it is a requirement in **experimental and theoretical physics**,

2

Why lectures on deep learning?

because

- it is an essential set of **algorithms** for **building models** in science,
- fast development of new **tools and algorithms** in the past years,
- nowadays it is a requirement in **experimental and theoretical physics**,
- useful set of skills for **industries** and **companies**.

2

Why deep learning for physics?

3

Why deep learning for physics?

- Predictive modeling.
- Ambiguous choices.
- Lack of information.
- Signal processing.
- Feature extraction.
- Data preprocessing.

3

Outline

Outline

Expected learning outcomes:

- illustrate deep learning models,
- identify deep learning models suitable for the resolution of specific problems,
- use software for the implementation of deep learning models.

4

Outline

Expected learning outcomes:

- illustrate deep learning models,
- identify deep learning models suitable for the resolution of specific problems,
- use software for the implementation of deep learning models.

Major topics:

- DL regressors and classifiers
- Hyper-optimization techniques
- Recursive Neural Networks
- Generative models
- Reinforcement learning
- Transfer Learning
- Object detection
- Image segmentation
- Graph Neural Networks
- Quantum Machine Learning

4

Some references

- *The elements of statistical learning*, T. Hastie, R. Tibshirani, J. Friedman.
- *An introduction to statistical learning*, G. James, D. Witten, T. Hastie, R. Tibshirani.
- *Deep learning*, I. Goodfellow, Y. Bengio, A. Courville.



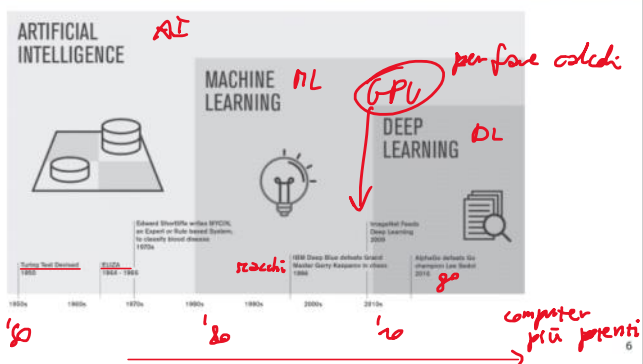
Bello

più moderno

5

Artificial Intelligence

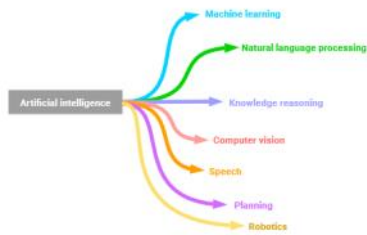
Artificial intelligence timeline



6

Defining A.I.

Artificial intelligence (A.I.) is the science and engineering of making intelligent machines.
(John McCarthy '56)



A.I. consist in the development of **computer systems** to perform tasks commonly associated with intelligence, such as **learning**.

7

A.I. and humans

There are **two** categories of **A.I. tasks**:

- **abstract and formal**: easy for computers but difficult for humans, e.g. play chess (IBM's Deep Blue 1997).
→ **Knowledge-based** approach to artificial intelligence.



8

A.I. and humans

There are **two** categories of **A.I. tasks**:

- **abstract and formal**: easy for computers but difficult for humans, e.g. play chess (IBM's Deep Blue 1997).
→ **Knowledge-based** approach to artificial intelligence.



- **intuitive for humans but hard to describe formally**:
e.g. recognizing faces in images or spoken words.
→ **Concept** capture and generalization



8

Facile come regole, ma difficile da "risolvere"

Task intuitive, difficili da descrivere matematicamente, ma gli umani riescono ad astrarre, generalizzare e capire, che è difficile da modellizzare su un pc con criteri matematici

A.I. technologies

Historically, the **knowledge-based** approach has not led to a major success with intuitive tasks for humans, because:

- requires human **supervision** and hard-coded **logical inference rules**.
- lacks of **representation learning** ability.

9

A.I. technologies

Historically, the *knowledge-based* approach has not led to a major success with intuitive tasks for humans, because:

- requires human *supervision* and hard-coded *logical inference rules*.
- lacks of *representation learning* ability.

Solution:

The A.I. system needs to *acquire its own knowledge*.
This capability is known as **machine learning (ML)**.
→ e.g. write a program which learns the task.



9

Quindi per evitare di specificarlo, si è pensato che il programma stesso impari autonomamente i concetti

Posso dire anche machine learning, ma alla fine sto parlando di modelli matematici, anche non lineari

Machine learning definition

Machine Learning definition (from T. Mitchell in 1998):

A computer program is said to *learn* from *experience E* with respect to some class of *tasks T* and *performance measure P*, if its performance on *T*, as measured by *P*, improves with experience *E*.

10

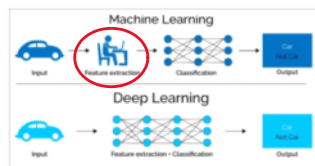
Machine learning definition

Machine Learning definition (from T. Mitchell in 1998):

A computer program is said to *learn* from *experience E* with respect to some class of *tasks T* and *performance measure P*, if its performance on *T*, as measured by *P*, improves with experience *E*.

Deep Learning

When a representation learning is difficult, ML provides **deep learning** techniques which allow the computer to build complex concepts out of simpler concepts, e.g. artificial neural networks.

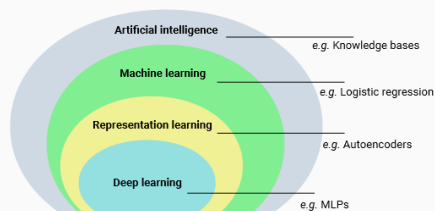


10

Definizione

La "differenza" di ML e DL, sta che prima facevo preprocessing, classificando e modificando i dati. In DL si hanno modelli abbastanza forti(e pesanti) da non aver più bisogno di preprocessing.

Venn diagram for A.I.



11

Machine learning examples

Thanks to work in A.I. and new capability for computers:

- Database mining:
 - Search engines
 - Spam filters
 - Medical and biological records



12

esempi:

Machine learning examples

Thanks to work in A.I. and new capability for computers:

- Database mining:
 - Search engines
 - Spam filters
 - Medical and biological records
- Intuitive tasks for humans:
 - Autonomous driving
 - Natural language processing
 - Robotics (reinforcement learning)
 - Game playing (DQN algorithms)



12

Machine learning examples

Thanks to work in A.I. and new capability for computers:

- Database mining:
 - Search engines
 - Spam filters
 - Medical and biological records
- Intuitive tasks for humans:
 - Autonomous driving
 - Natural language processing
 - Robotics (reinforcement learning)
 - Game playing (DQN algorithms)



12

Machine learning examples

Thanks to work in A.I. and new capability for computers:

- Database mining:
 - Search engines
 - Spam filters
 - Medical and biological records
- Intuitive tasks for humans:
 - Autonomous driving
 - Natural language processing
 - Robotics (reinforcement learning)
 - Game playing (DQN algorithms)
- Human learning:
 - Concept/human recognition
 - Computer vision
 - Product recommendation



12

ML applications in HEP

13

ML in experimental HEP

There are many applications in experimental HEP involving the **LHC measurements**, including the **Higgs discovery**, such as:

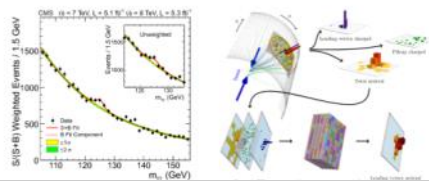
- Tracking
- Fast Simulation
- Particle identification
- Event filtering

14

ML in experimental HEP

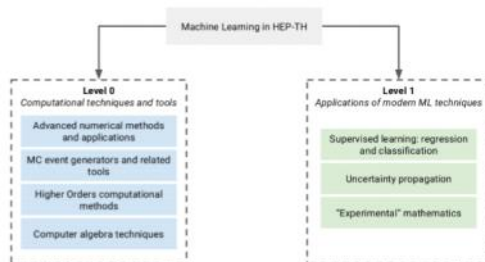
Some remarkable examples are:

- **Signal-background detection:**
Decision trees, artificial neural networks, support vector machines.
- **Jet discrimination:**
Deep learning imaging techniques via convolutional neural networks.
- **HEP detector simulation:**
Generative adversarial networks, e.g. LAGAN and CaloGAN.



15

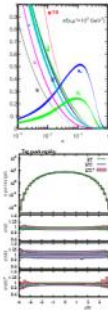
ML in theoretical HEP



16

ML in theoretical HEP

- **Supervised learning:**
 - The structure of the proton at the LHC
 - parton distribution functions
 - Theoretical prediction and combination
 - Monte Carlo reweighting techniques
 - neural network Sudakov
 - BSM searches and exclusion limits
- **Unsupervised learning:**
 - Clustering and compression
 - PDF4LHC15 recommendation
 - Density estimation and anomaly detection
 - Monte Carlo sampling
- **Reinforcement learning:**
 - Jet grooming



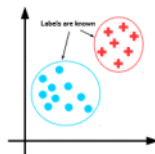
17

Learning paradigm

Machine learning algorithms

Machine learning algorithms:

- **Supervised learning:**
regression, classification, ...



Supervised learning



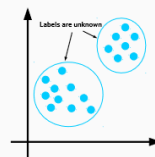
Il più comune e facile da implementare

18

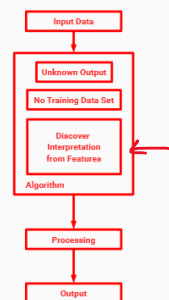
Machine learning algorithms

Machine learning algorithms:

- **Supervised learning:**
regression, classification, ...
- **Unsupervised learning:**
clustering, dim-reduction, ...



Unsupervised learning

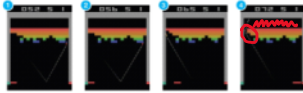


18

Machine learning algorithms

Machine learning algorithms:

- **Supervised learning:**
regression, classification, ...
- **Unsupervised learning:**
clustering, dim-reduction, ...
- **Reinforcement learning:**
real-time decisions, ...



Reinforcement learning



18

Machine learning algorithms



More than 60 algorithms.

19

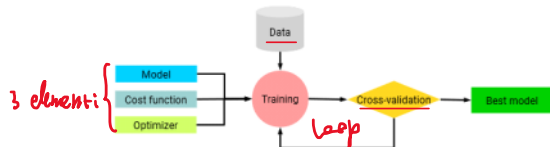
paradigm:



intro

Workflow in machine learning

The operative workflow in ML is summarized by the following steps:



The best model is then used to:

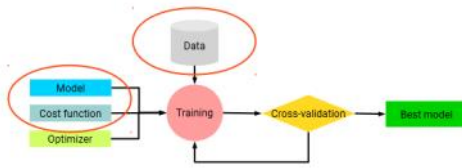
- supervised learning: make predictions for new observed data.
- unsupervised learning: extract features from the input data.

20

iniziamo da

Models and metrics

Models and metrics

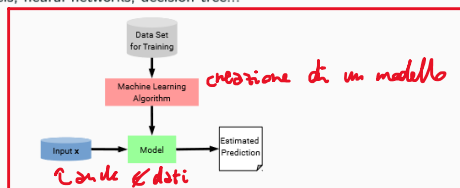


21

Model representation in supervised learning

We define parametric and structure models for statistical inference:

- examples: linear models, neural networks, decision tree...



- Given a training set of input-output pairs $A = (x_1, y_1), \dots, (x_n, y_n)$.
- Find a model $\mathcal{M}$ which:

$$\mathcal{M}(x) \sim y$$

where x is the input vector and y discrete labels in classification and real values in regression.

22

Model representation in supervised learning

Examples of models:

→ **linear regression** we define a vector $x \in \mathbb{R}^n$ as input and predict the value of a scalar $y \in \mathbb{R}$ as its output:

$$\hat{y}(x) = w^T x + b$$

where $w \in \mathbb{R}^n$ is a vector of parameters and b a constant.

23

Model representation in supervised learning

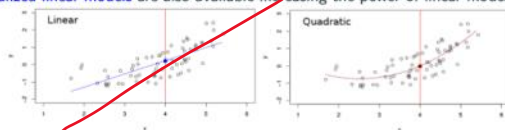
Examples of models:

→ **linear regression** we define a vector $x \in \mathbb{R}^n$ as input and predict the value of a scalar $y \in \mathbb{R}$ as its output:

$$\hat{y}(x) = w^T x + b$$

where $w \in \mathbb{R}^n$ is a vector of parameters and b a constant.

→ **Generalized linear models** are also available increasing the power of linear models:



23

Descriviamo matematicamente la questione

Model representation in supervised learning

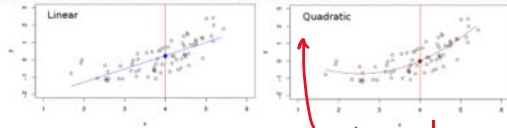
Examples of models:

→ **linear regression** we define a vector $x \in \mathbb{R}^n$ as input and predict the value of a scalar $y \in \mathbb{R}$ as its output:

$$\hat{y}(x) = w^T x + b$$

where $w \in \mathbb{R}^n$ is a vector of parameters and b a constant.

→ **Generalized linear models** are also available increasing the power of linear models:



→ **Non-linear models**: neural networks (talk later).

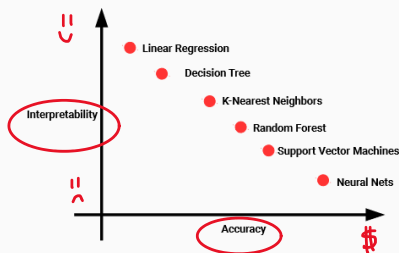
esempio:

Il problema è che devo definire a priori la funzione costo e il modello che sto usando. Devo quindi definire una performance metric adeguata

Model representation trade-offs

However, the selection of the appropriate model comes with **trade-offs**:

- **Prediction accuracy vs interpretability**:
→ e.g. linear model vs splines or neural networks.

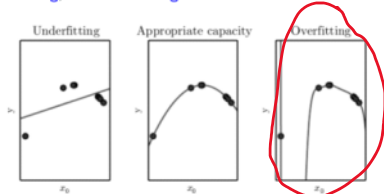


Trade-off: modello interpretabile --- risultato

Model representation trade-offs

However, the selection of the appropriate model comes with **trade-offs**:

- **Prediction accuracy vs interpretability**:
→ e.g. linear model vs splines or neural networks.
- **Optimal capacity/flexibility**: number of parameters, architecture
→ deal with **overfitting**, and **underfitting** situations



Problems

Devo decidere quanti e quali parametri usare, senza underfittare o overfittare

Assessing the model performance

How to check model performance?

→ define **metrics** and **statistical estimators** for **model performance**.

Examples:

- **Regression**: cost / loss / error function,
- **Classification**: cost function, precision, accuracy, recall, ROC, AUC

metriche regressione

metriche classificazione

Quindi al posto di fare un'analisi a mano, introduco delle metriche di performance

Assessing the model performance - cost function

To assess the model performance we define a **cost function** $J(w)$ which often measures the difference between the target and the model output.

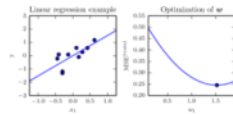
In a optimization procedure, given a model $\hat{y}_w$, we search for:

$$\arg \min_w J(w)$$

The **mean square error** (MSE) is the most commonly used for regression:

$$J(w) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_w(x_i))^2$$

a quadratic function and convex function in linear regression.



26

L'obiettivo è minimizzare la funzione di costo

esempio

Assessing the model performance - cost function

Other cost functions are depending on the nature of the problem.

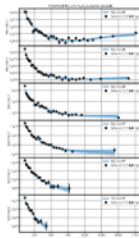
Some other examples:

- regression with uncertainties, **chi-square**:

$$J(w) = \sum_{i,j=1}^n (y_i - \hat{y}_w(x_i))(\sigma^{-1})_{ij}(y_j - \hat{y}_w(x_j))$$

where:

- σ_{ij} is the data covariance matrix.
e.g. for LHC data experimental statistical and systematics correlations.



27

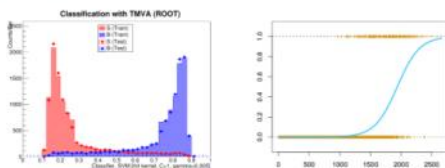
Cosa che useremo, chi quadro con 15:28, che tiene conto degli errori dei dati

Assessing the model performance - cost function

- logistic regression (binary classification): **cross-entropy**

$$J(w) = -\frac{1}{n} \sum_{i=1}^n y_i \log \hat{y}_w(x_i) + (1 - y_i) \log(1 - \hat{y}_w(x_i))$$

where $\hat{y}_w(x_i) = 1/(1 + e^{-w^T x_i})$.



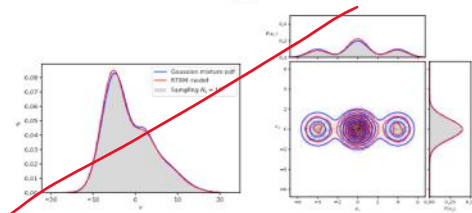
28

Cross-entropy, classificazione binaria (altra metrica)

Assessing the model performance - cost function

- density estimate / regression: **negative log-likelihood**:

$$J(w) = -\sum_{i=1}^n \log(\hat{y}_w(x_i))$$

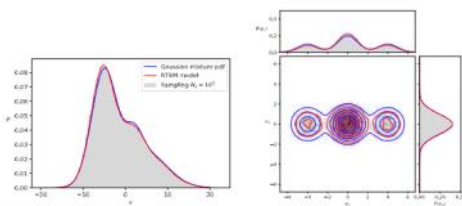


29

Assessing the model performance - cost function

- density estimate / regression: **negative log-likelihood**:

$$J(w) = - \sum_{i=1}^{n_1} \log(\hat{y}_w(x_i))$$



- Kullback-Leibler, RMSE, MAE, etc.

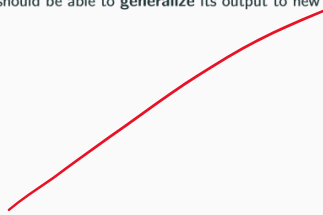
29

Negative log likelihood, se devo predire la forma, la loss function sarà quindi migliore nella regione interessata

Training and test sets

Another common issue related to model capacity in supervised learning:

- The model should not learn **noise** from data.
- The model should be able to **generalize** its output to new samples.



30

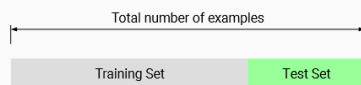
Training and test sets

Another common issue related to model capacity in supervised learning:

- The model should not learn **noise** from data.
- The model should be able to **generalize** its output to new samples.

To observe this issue we split the input data in training and test sets:

- training set error**, $J_{Tr}(w)$
- test set/generalization error**, $J_{Test}(w)$



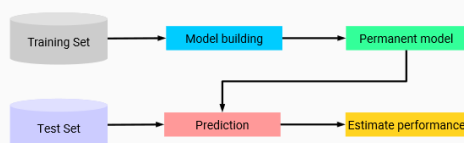
30

Come capisco se ho il modello migliore?

Per farlo divido i dati, in train e test, in questo modo posso misurare la performance sia su dati conosciuti, che sconosciuti, ho quindi un generalized error

Training and test sets

The test set is independent from the training set but follows the same probability distribution.



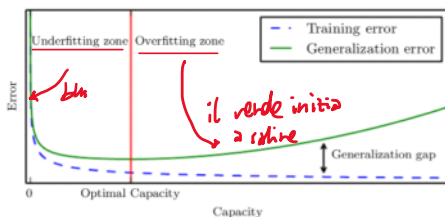
31

Processo di controllo

K-fold → test diversità sufficiente
n: 3

Bias-variance trade-off

From a practical point of view dividing the input data in training and test:



The training and test/generalization error conflict is known as **bias-variance trade-off**.

Quindi siccome l'errore di generalizzazione inizia a salire, il significato è che sto imparando a memoria i dati, e sto perdendo la capacità di generalizzare

Bias/Variance trade-off

Bias-variance trade-off

Supposing we have model $\hat{y}(x)$ determined from a training data set, and considering as the true model

$$Y = y(X) + \epsilon, \text{ with } y(x) = E(Y|X = x),$$

where the noise ϵ has zero mean and constant variance.



Bias-variance trade-off

Supposing we have model $\hat{y}(x)$ determined from a training data set, and considering as the true model

$$Y = y(X) + \epsilon, \text{ with } y(x) = E(Y|X = x),$$

where the noise ϵ has zero mean and constant variance.

If we take (x_0, y_0) from the test set then:

$$E[(y_0 - \hat{y}(x_0))^2] = (\text{Bias}[\hat{y}(x_0)])^2 + \text{Var}[\hat{y}(x_0)] + \text{Var}(\epsilon),$$

where

- $\text{Bias}[\hat{y}(x_0)] = E[\hat{y}(x_0)] - y(x_0)$
- $\text{Var}[\hat{y}(x_0)] = E[\hat{y}(x_0)^2] - (E[\hat{y}(x_0)])^2$

So, the expectation averages over the variability of y_0 (bias) and the variability in the training data.

Un bias alto vuol dire che la predizione è fuori dal contesto, mentre la varianza sta misurando quanto fluttua il modello, se ho troppa varianza sto imparando i dati

Bias-variance trade-off

If $\hat{y}$ increases flexibility, its **variance increases** and its **bias decreases**.

Choosing the flexibility based on average test error amounts to a **bias-variance trade-off**:

- **High Bias** → underfitting: erroneous assumptions in the learning algorithm.
- **High Variance** → overfitting: erroneous sensitivity to small fluctuations (noise) in the training set.

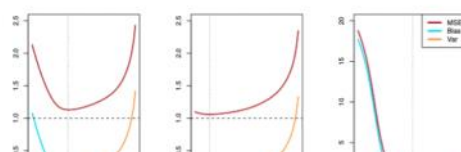


Quindi con poche variabili underfitto e ho bias, mentre se ho molte variabili imparo troppo, anche l'errore.

Il problema di questo grafico è che non posso usarlo in pratica perché non conosco il true low

Bias-variance trade-off

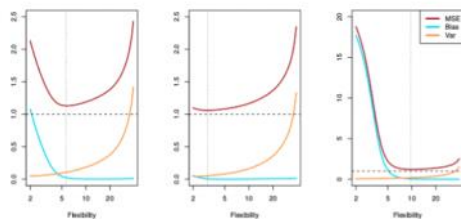
More examples of bias-variance trade-off:



Ho quindi un trade-off sulla capacità

Bias-variance trade-off

More examples of bias-variance trade-off:



35

Ho quindi un trade-off sulla capacità

Bias-variance trade off

Regularization techniques can be applied to modify the learning algorithm and **reduce** its generalization error but not its training error.

For example, including the **weight decay** to the MSE cost function:

$$J(w) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_w(x_i))^2 + \lambda w^T w$$

parametro a mano

where λ is a real number which express the preference for weights with smaller squared L^2 norm.

36

Come aggiusto questo problema?

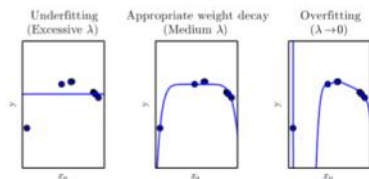
Posso ridurre le fluttuazioni applicando tecniche di regolarizzazione, quindi posso attaccare un pezzo che penalizza il modello se scelgo parametri grossi.

Si può mostrare che se la somma dei parametri quadrati sono alti, ho molto rumore dentro.

Mettere il termine blu previene termini grandi.

Solution for the bias-variance trade off

Tuning the hyperparameter λ we can regularize a model without modifying explicitly its capacity.



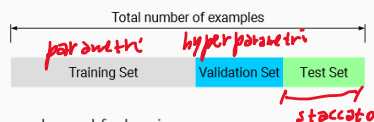
37

Mettendo hyperparametri, che evitano problemi a livello di fit, creano un nuovo problema, perché devo "fittarli", trovarne dei valori buoni

Solution for the bias-variance trade off

A common way to reduce the bias-variance trade-off and choose the proper learning hyperparameters is to create a **validation** set that:

- not used by the training algorithm
- not used as test set



- **Training set:** examples used for learning.
- **Validation set:** examples used to tune the hyperparameters.
- **Test set:** examples used only to assess the performance.

Techniques are available to deal with data samples with large and small number of examples. (talk later)

38

In realtà non ho solo training e test, ma se uso il test per determinare la performance, quindi non sempre posso usarlo da solo per determinare la bontà del modello.

Se ho dati disponibili, posso splittarlo in 3. La validation la uso per ottimizzare gli iperparametri, e il test viene usato solo alla fine per rankare i modelli migliori.

Assessing model performance for classification

In binary classification tasks we usually complement the cost function with the **accuracy** metric defined as:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Example:

True Positives (TP) e.g. 8	False Positives (FP) e.g. 2
False Negatives (FN) e.g. 4	True Negatives (TN) e.g. 20

- Accuracy = 82%

39

Assessing model performance for classification

In binary classification tasks we usually complement the cost function with the **accuracy** metric defined as:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Example:

True Positives (TP) e.g. 8	False Positives (FP) e.g. 2
False Negatives (FN) e.g. 4	True Negatives (TN) e.g. 20

- Accuracy = 82%

However accuracy does not represent the overall situation for skewed classes, i.e. imbalance data set with large disparity, e.g. **signal and background**.

In these cases we define **precision** and **recall**.

39

Assessing model performance for classification

Precision: proportion of correct positive identifications.

Recall: proportion of correct actual positives identifications.

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}$$

True Positives (TP) e.g. 8	False Positives (FP) e.g. 2
False Negatives (FN) e.g. 4	True Negatives (TN) e.g. 20

- Accuracy = 82%
- Precision = 80%
- Recall = 67%

40

Assessing model performance for classification

Precision: proportion of correct positive identifications.

Recall: proportion of correct actual positives identifications.

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}$$

True Positives (TP) e.g. 8	False Positives (FP) e.g. 2
False Negatives (FN) e.g. 4	True Negatives (TN) e.g. 20

- Accuracy = 82%
- Precision = 80%
- Recall = 67%

Various metrics have been developed that rely on both precision and recall, e.g. the F_1 score:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = 73\%$$

40

Se faccio classificazione ho ulteriori problemi.
Posso usare una matrice di confusione.

A seconda di quanto posso sbagliare posso avere diverse metriche. Per esempio accuracy, precision e recall

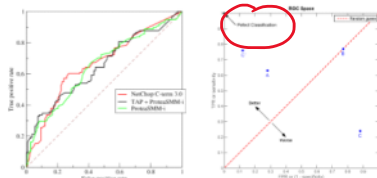
Questa mette anche insieme varie metriche

Assessing model performance for classification

In a binary classification we can vary the probability threshold and define:

- the receiver operating characteristic curve (ROC curve) is a metric which shows the relationship between correctly classified positive cases, the true positive rate (TRP/recall) and the incorrectly classified negative cases, false positive rate (FPR, (1-effectivity)).

$$TPR = \frac{TP}{TP + FN}, \quad FPR = \frac{FP}{FP + TN}$$



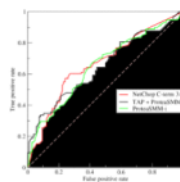
41

Assessing model performance for classification

The area under the ROC curve (AUC) represents the probability that classifier will rank a randomly chosen positive instance higher than a randomly chosen negative one.

AUC provides an aggregate measure of performance across all possible classification thresholds.

- AUC is 0 if predictions are 100% wrong
- AUC is 1 if all predictions are correct.
- AUC is scale-invariant and classification-threshold-invariant.



42

Summary

Summary

We have covered the following topics:

- Motivation and overview of A.I.
- Definition and overview of ML.
- Model representation definition and trade-offs
- Learning metrics for accessing the model performance
- Metrics for classification.

43

Esistono anche metriche più intelligenti. Creando ROC posso fare variazioni nel dataset e plottare le variazioni ...

La curva più vicino alto e sinistra è quella ideale, per esempio la rossa sembra la migliore

Per esempio per capire il migliore posso integrare l'area sotto la ROC e prendo quella con area maggiore

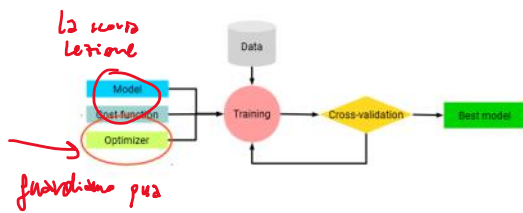
Deep Learning - lecture 2

Algorithms and techniques for parameter learning

Prof. Stefano Carrazza

University of Milan and INFN Milan

Parameter learning

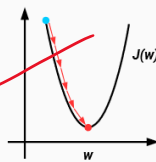


1

Cost function minimization

Optimization algorithms minimize an objective function, $J(w)$, that depends on the model internal learnable parameters w .

$$\arg \min_w J(w)$$



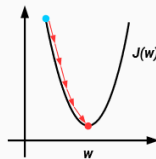
2

Cost function minimization

Optimization algorithms minimize an objective function, $J(w)$, that depends on the model internal learnable parameters w .

$$\arg \min_w J(w)$$

et di parametri
cost



The most popular techniques are:

- normal equations (least squares)
 - derivative based optimization
 - evolutionary algorithms
- derivare* (pointing to derivative based optimization)
perché (pointing to evolutionary algorithms)
- multi dati / multi parametri* (grouped with the first two items by a red bracket)

The choice of a technique depends on the model and problem employed.

2

Vogliamo minimizzare la loss function

Normal equations

The **normal equation** is a method to solve for w analytically.

- it is employed in linear and non-linear **least squares** optimization.
- it is fast for small models with few features, otherwise it can be computationally intensive and **slow**.

3

Vogliamo risolvere analiticamente il problema

Normal equations example

Example: multivariate linear regression.

Suppose we have m training examples and n features in a matrix X , size (m, n) , and the observed values y we have to solve the system:

$$Xw = y$$

How to solve it when the system is overdetermined?

4

Normal equations example

Example: multivariate linear regression.

Suppose we have m training examples and n features in a matrix X , size (m, n) , and the observed values y we have to solve the system:

$$Xw = y$$

How to solve it when the system is overdetermined?

1. Define the cost function for the problem:

$$J(w) = \|y - Xw\|^2$$

4

Normal equations example

Example: multivariate linear regression.

Suppose we have m training examples and n features in a matrix X , size (m, n) , and the observed values y we have to solve the system:

$$Xw = y$$

How to solve it when the system is overdetermined?

1. Define the cost function for the problem:

$$J(w) = \|y - Xw\|^2$$

2. Compute and impose:

$$\frac{\partial J(w)}{\partial w} = 0$$

4

Normal equations example

Example: multivariate linear regression.

Suppose we have m training examples and n features in a matrix X , size (m, n) , and the observed values y we have to solve the system:

$$Xw = y$$

How to solve it when the system is overdetermined?

1. Define the cost function for the problem:

$$J(w) = \|y - Xw\|^2$$

2. Compute and impose:

$$\frac{\partial J(w)}{\partial w} = 0$$

3. We obtain the solution:

$$w = (X^T X)^{-1} X^T y = X^+ y$$

with X^+ is the pseudoinverse of X .

*↑ ← features
examples*

*↓ solve
analytically*

Formula chiusa

4

normal equations

Processo di diagonalizzazione della matrice

Problema: posso avere molti dati, e la matrice diventa molto grossa

Se la X ha tanti dati m o features n , la complessità dell'inversione può essere molto pesante

Derivative based optimization

More general optimization algorithms based on derivatives:

- **First order optimization algorithms:** uses the gradient of the cost function with respect to the parameters in an iterative procedure.
→ **gradient descent** algorithms.

5

Derivative based optimization

More general optimization algorithms based on derivatives:

- **First order optimization algorithms:** uses the gradient of the cost function with respect to the parameters in an iterative procedure.
→ **gradient descent** algorithms.
- **Second order optimization algorithms:** uses the Hessian of the cost function and takes care of the curvature of surface.
→ if the Hessian is known it may be faster than gradient descent,
→ otherwise slow due to the Hessian evaluation.

5

Derivative based optimization

More general optimization algorithms based on derivatives:

- **First order optimization algorithms:** uses the gradient of the cost function with respect to the parameters in an iterative procedure.
→ **gradient descent** algorithms.
- **Second order optimization algorithms:** uses the Hessian of the cost function and takes care of the curvature of surface.
→ if the Hessian is known it may be faster than gradient descent,
→ otherwise slow due to the Hessian evaluation.

In practice **Gradient Descent** is the most popular technique in ML.

vince il primo ordine

5

Alternative sono:

Ho la loss function, calcolo il gradiente e lo seguo, andando a cercare il minimo. Questi algoritmi sono detti di **gradient descent**

Lavorare con derivate prime non ci dice però se il nostro minimo è locale o anche globale.

Se quindi uso la derivata seconda per capire quali sono i minimi, può essere che la derivata seconda è molto pesante da calcolare (rispetto le derivate al primo ordine). Quindi anche se è la scelta matematica corretta, può essere troppo pesante

Which method?

Q: normal equation or derivative based?

Suppose we have m training examples and n features.

Normal equation

- ✓ no parameters to tune
- ✓ no iterations
- ✗ slow if n is large
- ✗ requires $(X^T X)^{-1}$, $\mathcal{O}(n^3)$

6

Which method?

Q: normal equation or derivative based?

Suppose we have m training examples and n features.

Normal equation

- ✓ no parameters to tune
- ✓ no iterations
- ✗ slow if n is large
- ✗ requires $(X^T X)^{-1}$, $\mathcal{O}(n^3)$

Gradient descent

- ✓ efficient when n is large
- ✓ easy to implement/use
- ✗ requires iterations
- ✗ requires parameter tune

← punto delicato

Paragone equazioni normali e gradient Descent

6

Gradient descent idea

Basic idea:

Assuming we want to minimize $J(w)$ where w is a vector of parameters:

- select a initial solution vector w ,
- change the w to reduce $J(w)$

Repeat until a minimum of $J(w)$ is reached.

7

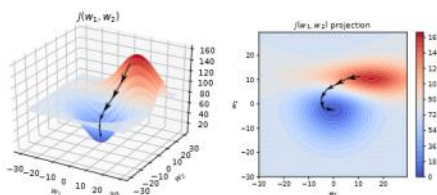
Gradient descent idea

Basic idea:

Assuming we want to minimize $J(w)$ where w is a vector of parameters:

- select a initial solution vector w ,
- change the w to reduce $J(w)$ ↻

Repeat until a minimum of $J(w)$ is reached.



7

Gradient descent algorithm

Simultaneously for each parameter in w repeat until convergence:

$$w_i := w_i - \eta \frac{\partial}{\partial w_i} J(w)$$

where η is the learning rate, $\eta \geq 0$, it can be a fixed number, because the gradient term will automatically compensate with smaller steps: $\nabla_w J|_{w \rightarrow w_{\text{best}}} \rightarrow 0$

8

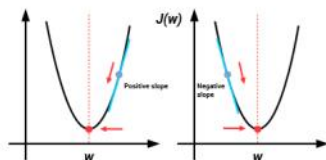
Gradient descent algorithm

Simultaneously for each parameter in w repeat until convergence:

$$w_i := w_i - \eta \frac{\partial}{\partial w_i} J(w)$$

where η is the learning rate, $\eta \geq 0$, it can be a fixed number, because the gradient term will automatically compensate with smaller steps: $\nabla_w J|_{w \rightarrow w_{\text{best}}} \rightarrow 0$

Why the negative sign in term $-\eta$? (example in 1D)



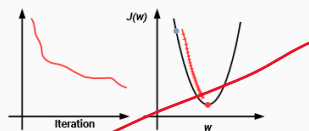
- if $\nabla_w J(w) > 0$ then w decreases
- if $\nabla_w J(w) < 0$ then w increases

8

Gradient descent and learning rate

The η is another example of hyperparameter which requires tune.

- if η is **too small**, gradient descent can be slow.

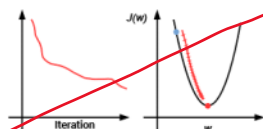


9

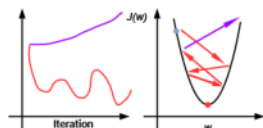
Gradient descent and learning rate

The η is another example of hyperparameter which requires tune.

- if η is **too small**, gradient descent can be slow.



- if η is **too large**, gradient descent may **fail to converge or diverge**.



9

La tecnica è semplice. Dati i parametri iniziali w , ho che lo step dopo w_i è aggiornato come

Usando il learning rate fa sì che i primi passi sono più grezzi, e pian piano li rallento. Ma al contrario devo sia settare la velocità iniziale che il come decellera.

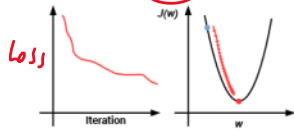
Ho un - perché voglio puntare al minimo

Non posso sapere se il minimo è globale. Uso un criterio di qualità. Se il valore è accettabile, se non è sufficiente devo cambiare metodo

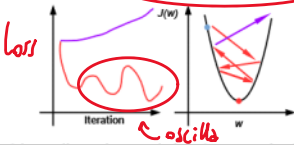
Gradient descent and learning rate

The η is another example of hyperparameter which requires tune.

- if η is too small, gradient descent can be slow.



- if η is too large, gradient descent may fail to converge or diverge.



η è un nuovo iperparametro che devo aggiustare

Alcuni problemi se eta è settato male

Quindi guardo la forma della loss. Se oscilla è troppo grande. Se va giù troppo piano è troppo piccolo

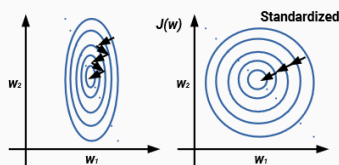
Gradient descent and feature scaling

Another practical hint: feature scaling.

Make sure the input features x_i are in a similar scale, e.g. standardization:

$$\tilde{x}_i := \frac{x_i - \mu_{x_i}}{\sigma_{x_i}}$$

where μ_{x_i} and σ_{x_i} are the mean and standard deviation respectively.



Un altro discorso è sui dati. Se ho un dataset con scale diverse su diverse features, la loss function potrebbe essere squeezeata.

Questo scenario si risolve usando una standardizzazione dei dati.

I dati entrano nella loss function, e quindi se hanno ordini di grandezza diversi fanno casini.

Gradient descent variants

When performing gradient descent the cost function $J(w)$ is evaluated over the training data, e.g. for the MSE cost function:

$$\frac{\partial}{\partial w} J(w) = \frac{\partial}{\partial w} \left(\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_w(x_i))^2 \right)$$

If the training data set is too large, there are gradient descent variations that may improve convergence in terms of speed and quality:

- Batch Gradient Descent:** all training data points are evaluated in the cost function gradient at each iteration.

Devo fare la somma per tutte le derivate, diventa quindi molto pesante.

Lavoro con tutti i dati?
Ho diverse tecniche.

- **Batch gradient descent:** Sì, li uso tutti, se ho dataset non troppo grossi, che possono entrare in memoria
- Con il SGD: uso UN solo punto, e lo uso per calcolare il gradiente, esatto opposto. Ma potenzialmente può raggiungere nuove minimi(?).
- **Mini Batch Gradient Descent:** è una via di mezzo, ed è quella che si usa di solito. Quindi al posto di lavorare con tutti i dati, definiamo una lunghezza massima di elementi che uso nel gradient descent, la **batch size**, che viene usato ad ogni iterazione (posso usare anche shuffling). Permette di tenere più sotto controllo le varianze.

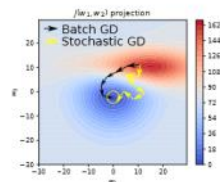
Gradient descent variants

- Stochastic Gradient Descent (SGD):**

- randomly shuffle training examples,
- use 1 example at each iteration.

Features:

- parameters updates have high variance and cost function fluctuates.
- helps to discover new and possibly better minima.
- requires to slowly decrease the learning rate η to reduce fluctuations.



più
casuale

N.B. la SGD è normalmente implementata con un batch size, e non è per forza a singolo parametro

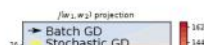
Gradient descent variants

- Mini Batch Gradient Descent:**

- use a subset of size b (batch size) of examples in each iteration,
- use the batch set example at each iteration.

Features:

- takes the best from both previous methods,
- reduces the variance in the parameter updates (stable convergence),
- good for data parallelism, efficient for matrix operations



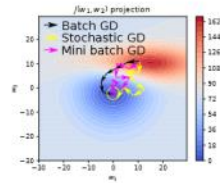
Gradient descent variants

- **Mini Batch Gradient Descent:**

1. use a subset of size b (batch size) of examples in each iteration,
2. use the batch set example at each iteration.

Features:

- takes the best from both previous methods,
- reduces the variance in the parameter updates (stable convergence),
- good for data parallelism, efficient for matrix operations



13

Gradient descent schemes

SGD has many improvements and extensions, for example:

- **Momentum:** it stores the update Δw at each iteration and update parameters following:

$$w := w - \eta \nabla_w J(w) + \alpha \Delta w$$

14

Gradient descent schemes

SGD has many improvements and extensions, for example:

- **Momentum:** it stores the update Δw at each iteration and update parameters following:

$$w := w - \eta \nabla_w J(w) + \alpha \Delta w$$

- **Root Mean Square Propagation (RMSProp):** it introduces an adaptive learning rate for each parameter:

$$w := w - \frac{\eta}{\sqrt{v(w, ite)}} \nabla_w J(w)$$

14

Gradient descent schemes

SGD has many improvements and extensions, for example:

- **Momentum:** it stores the update Δw at each iteration and update parameters following:

$$w := w - \eta \nabla_w J(w) + \alpha \Delta w$$

- **Root Mean Square Propagation (RMSProp):** it introduces an adaptive learning rate for each parameter:

$$w := w - \frac{\eta}{\sqrt{v(w, ite)}} \nabla_w J(w)$$

- Others: **Averaging**, **AdaGrad**, **Adam**, etc...

14

Gradient descent schemes

SGD has many improvements and extensions, for example:

- **Momentum**: it stores the update Δw at each iteration and update parameters following:

$$w := w - \eta \nabla_w J(w) + \alpha \Delta w$$

- **Root Mean Square Propagation** (RMSProp): it introduces an adaptive learning rate for each parameter:

$$w := w - \frac{\eta}{\sqrt{v(w, ite)}} \nabla_w J(w)$$

altre

- Others: **Averaging**, **AdaGrad**, **Adam**, etc...

All these schemes and respective parameters can be considered as extra hyperparameters to tune.

14

Posso variare lo schema di gradient descent

Buono se ho tanti punti di sella. Permette di scavalcare selle

Posso volere learning rate diversi per diversi features, il learning rate viene ora normalizzato da una funzione che tiene conto sia del numero dell'iterazione, che del parametro

Examples of second order optimization

Popular examples of second order optimization algorithms:

- **Newton's method**: an iterative method based on Taylor expansion.

Example in 1D: consider the Taylor expansion

$$J_T(w) = J_T(w_n + \Delta w) \approx J(w_n) + J'(w_n)\Delta w + \frac{1}{2}J''(w_n)\Delta w^2$$

We aim to find Δw which satisfies:

$$\nabla_{\Delta w} J_T(w_n + \Delta w) = J'(w_n) + J''(w_n)\Delta w = 0$$

$$w_{n+1} = w_n + \Delta w = w_n - \frac{J'(w_n)}{J''(w_n)}$$

$w_1, w_2, \dots$ will converge to a stationary point w^* where $J'(w^*) = 0$.

15

E tutti queste tecniche con altri iperparametri, dovranno poi essere fine tunati. Quali con più consistenza raggiungono minimi, e questi minimi sono buoni.

Vogliamo schemi che ci porti a un miglior numero di tentativi in cui non si blocca la loss

Memoria vs performance (in termini di runtime)(se posso è più efficiente un full bach)
Poi il discorso di risultati è più complicato

Examples of second order optimization

Popular examples of second order optimization algorithms:

- **Newton's method**: an iterative method based on Taylor expansion.

Example in 1D: consider the Taylor expansion

$$J_T(w) = J_T(w_n + \Delta w) \approx J(w_n) + J'(w_n)\Delta w + \frac{1}{2}J''(w_n)\Delta w^2$$

We aim to find Δw which satisfies:

$$\nabla_{\Delta w} J_T(w_n + \Delta w) = J'(w_n) + J''(w_n)\Delta w = 0$$

$$w_{n+1} = w_n + \Delta w = w_n - \frac{J'(w_n)}{J''(w_n)}$$

$w_1, w_2, \dots$ will converge to a stationary point w^* where $J'(w^*) = 0$.

Generalization in N dimensions:

$$(H J(w_n)) \Delta w = -\nabla J(w_n)$$

where H is the Hessian matrix.

15

Quelli che usano la derivata seconda sono interessanti

low e derivate

difficile da applicare

Examples of second order optimization

Popular examples of second order optimization algorithms:

- **Quasi-newton methods**: i.e. methods which optimizes even if the Hessian matrix is expensive or not available. The Taylor's series is:

$$J(w_n + \Delta w) \approx J(w_n) + \nabla_w J(w_n)^T \Delta w + \frac{1}{2} \Delta w^T B \Delta w,$$

where B is an approximation to the Hessian matrix and

$$\nabla_w J(w_n + \Delta w) \approx \nabla_w J(w_n) + B \Delta w,$$

which produces the **Newton step**:

$$\Delta w = -B^{-1} \nabla_w J(w_n).$$

Some methods: BFGS, L-BFGS, DFP, Broyden.

- differ by the choice of the solution to update B .

Popular in ML since the beginning of the deep learning era.

16

Alcuni metodi evitano di calcolare l'Hessiana, usando trucchi, e imitando il metodo di Newton

dipende dal metodo

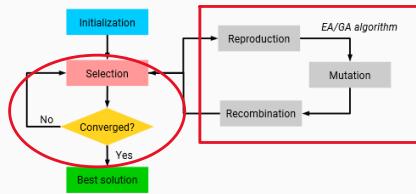
Buoni per modelli piccoli, in generale chiedono però troppa memoria e tempo di calcolo

Evolutionary algorithms

Evolutionary algorithms (EA), inspired by biological evolution, is a generic population-based metaheuristic optimization algorithm.

Techniques in EA use mechanisms such as: reproduction, mutation, recombination, and selection.

Genetic algorithm is the most popular technique of EA.



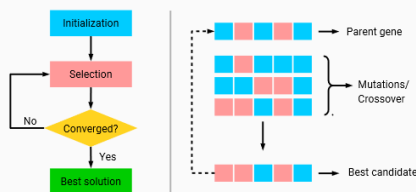
17

Genetic algorithm

Genetic algorithm is well suited when:

- gradients are not available,
- non parametric function,
- non homogeneous cost function along all training set points.

Example:



18

Summary

Summary

We have covered the following topics:

- Parameter learning: normal equations, first and second order optimization, genetic algorithm

→ gradient descent
quello usato quasi sempre

19

Usati molto prima degli schemi di gradient descent. Per modelli grossi, la versione vanilla non è il massimo. Molto facilmente si bloccano su minimi locali, e c'era molta apprensione a riguardo. Una ventina d'anni fa si usavano quindi tecniche un po' più brute force

Questo modello si ispira alla genetica, codificando il modello con geni, che possono subire varie modifiche. Si calcola poi la loss function e si prende il migliore.

Questo è molto inefficiente, devo fare sia molte copie, che calcolare anche molte volte la loss function. Inoltre è una procedura molto stocastica, le mutazioni sono abbastanza casuali, a differenza del gradiente.

Ma in alcuni casi è l'unico modo.

Tra le applicazioni del quantum computing, il problema è che gli esperimenti sono molto rumorosi. Se devo usare il chip come modello il gradiente è troppo rumoroso. Invece con modelli genetici trova la configurazione rumorosa migliore per il chip.

Una loss function che non è parametrica, significa che la predizione non ha una scrittura analitica, ma catene di eventi che producono un qualcosa

All'epoca si usavano in questo caso

A livello di hyperparametri, non ho una loss function parametrica. In questo caso può essere una buona idea usare algoritmi genetici per trovare i parametri migliori. Che è un esempio di non parametric function

Ibridi tra GA e altre tecniche

Lo state of the art per gli hparam smv? Che può essere approssimata da un modello bayesiano, che capisce la distribuzione di probabilità e ci dà cambi più furbi in media

Deep Learning - lecture 3

Non-linear models

Prof. Stefano Carrazza

University of Milan and INFN Milan

Artificial neural networks

Limitations of linear models

Why not linear models everywhere?



1

Limitations of linear models

Why not linear models everywhere?

Example: consider 1 image from the MNIST database:



Each image has 28×28 pixels = 785 features (x3 if including RGB colors).

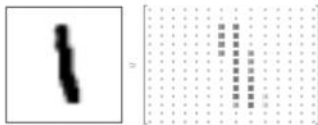
If consider quadratic function $\mathcal{O}(n^2)$ so linear models are impractical.

1

Limitations of linear models

Why not linear models everywhere?

Example: consider 1 image from the MNIST database:



Each image has 28×28 pixels = 785 features (x3 if including RGB colors).

If consider quadratic function $O(n^2)$ so linear models are impractical.

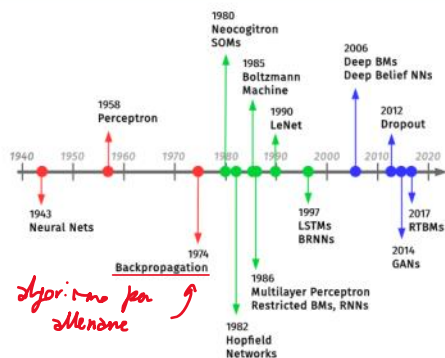
Solution: use non-linear models.

1

Cose non lineari nascono per descrivere cose più complicate, per esempio immagini

La soluzione è quindi l'uso di modelli non lineari

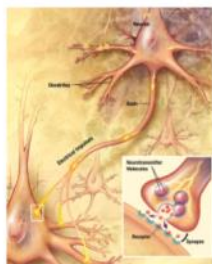
Non-linear models timeline



2

Neural networks

Artificial neural networks are computer systems inspired by the biological neural networks in the brain.



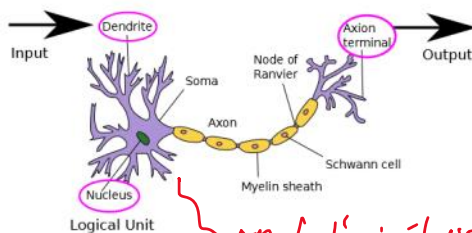
Currently the state-of-the-art technique for several ML applications.

3

Dagli studi sul cervello, si è provato a copiare il modello neurale

Neuron model

We can imagine the following data communication pattern:



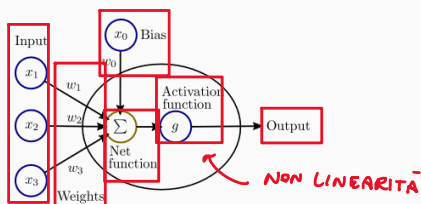
cosa fa l'unità logica, il neurone?

4

Funzionamento del neurone

Neuron model

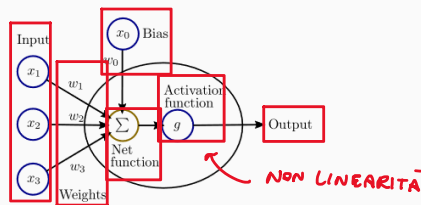
Schematically:



Rappresentazione matematica analoga

Neuron model

Schematically:



where

- each **node** has an associate weights and bias w and inputs x ,
- the output is modulated by an **activation function**, g .

Some examples of activation functions: sigmoid, tanh, linear, ...

$$g_w(x) = \frac{1}{1 + e^{-w^T x}}, \quad \tanh(w^T x), \quad x.$$

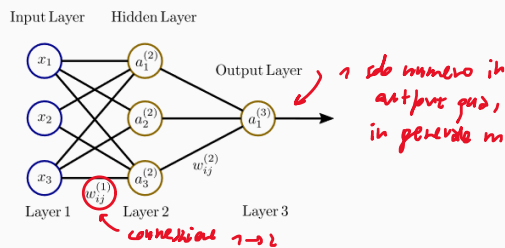
5

Rappresentazione matematica analoga

Neural networks

In practice, we simplify the bias term with $x_0 = 1$.

Neural network → connecting multiple units together.



where

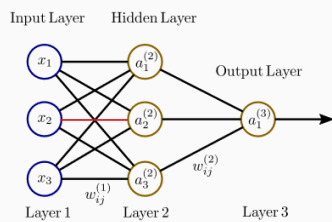
- $a_i^{(l)}$ is the activation of unit i in layer l ,
- $w_{ij}^{(l)}$ is the weight between nodes i, j from layers $l, l+1$ respectively.

6

L'unione di neuroni, è una rete

← #layer
2, 1
← #node

Neural networks



- $a_1^{(2)} = g(w_{10}^{(1)} + w_{11}^{(1)}x_1 + w_{12}^{(1)}x_2 + w_{13}^{(1)}x_3)$
- $a_2^{(2)} = g(w_{20}^{(1)} + w_{21}^{(1)}x_1 + w_{22}^{(1)}x_2 + w_{23}^{(1)}x_3)$
- $a_3^{(2)} = g(w_{30}^{(1)} + w_{31}^{(1)}x_1 + w_{32}^{(1)}x_2 + w_{33}^{(1)}x_3)$
- Output** → $a_1^{(3)} = g(w_{10}^{(2)} + w_{11}^{(2)}a_1^{(2)} + w_{12}^{(2)}a_2^{(2)} + w_{13}^{(2)}a_3^{(2)})$

7

forward pass

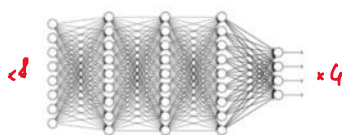
In problemi di regressione, se uso non linearità non posso raggiungere tutti i punti in $\mathbb{R}$, quindi in quel caso di solito l'ultimo layer è linear

Neural networks

Some useful names:

• **Dense Layer**

- Feedforward neural network**: no cyclic connections between nodes from the same layer (previous example).
- Multilayer perceptron (MLP)**: is a feedforward neural network with at least 3 layers.
- Deep neural networks**: term referring to neural networks with more than one hidden layer.



8

Nomi

Training neural networks

The training NNs is usually performed with **gradient descent** methods.

Following the previous section, we have to compute the cost function gradient with respect to parameters $w_{ij}^{(l)}$:

$$w_{ij}^{(l)} := w_{ij}^{(l)} - \eta \nabla_{ij}^{(l)} J \rightarrow \nabla_{ij}^{(l)} J = \frac{\partial}{\partial w_{ij}^{(l)}} J(w)$$

9

Training neural networks

The training NNs is usually performed with **gradient descent** methods.

Following the previous section, we have to compute the cost function gradient with respect to parameters $w_{ij}^{(l)}$:

$$w_{ij}^{(l)} := w_{ij}^{(l)} - \eta \nabla_{ij}^{(l)} J \rightarrow \nabla_{ij}^{(l)} J = \frac{\partial}{\partial w_{ij}^{(l)}} J(w) \quad \forall i, j$$

Use the **backpropagation algorithm** to compute the gradient of a NN.

- can be used with any gradient-based optimizer, including quasi-Newton methods.
- reduces the large amount of computations thanks to chain rule
- requires the derivative of the cost function with respect to the output layer $w_{ij}^{(l)}$ with $l =$ output.

9

Training

evita il calcolo esplicito di tutto

Backpropagation algorithm

The backpropagation steps:

- 1: perform a **forward propagation** (calculate $a_i^{(l)}$)

10

Backpropagation algorithm

The backpropagation steps:

- 1: perform a **forward propagation** (calculate $a_i^{(l)}$)
- 2: perform a **backward propagation**: evaluate for each node a "prediction error":

$$\delta_j^{(l)} = \text{"error" of node } j \text{ in layer } l.$$

10

Backpropagation algorithm

The backpropagation steps:

- 1: perform a **forward propagation** (calculate $a_i^{(l)}$)
- 2: perform a **backward propagation**: evaluate for each node a "prediction error":

$$\delta_j^{(l)} = \text{"error" of node } j \text{ in layer } l.$$

- 3: calculate $\nabla_{ij}^{(l)} J$ using errors $\delta_j^{(l)}$ and $a_i^{(l)}$.

10

Backpropagation algorithm

The backpropagation steps:

- 1: perform a **forward propagation** (calculate $a_i^{(l)}$)
- 2: perform a **backward propagation**: evaluate for each node a "prediction error":

$$\delta_j^{(l)} = \text{"error" of node } j \text{ in layer } l.$$

- 3: calculate $\nabla_{ij}^{(l)} J$ using errors $\delta_j^{(l)}$ and $a_i^{(l)}$.
- 4: perform **weight updates**, $\Delta w_{ij}^{(l)}$, via gradient descent using $\nabla_{ij}^{(l)} J$.

10

calcolo il feedforward

calcolo un errore partendo dall'ultimo

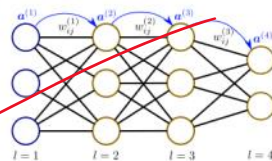
*quindi sento
la sua derivata
tra $\delta_i^{(l)}$ e $a_i^{(l)}$*

Backpropagation algorithm

Suppose we have a MLP and one training example (x, y) .

Step 1: We first perform a **forward propagation pass**:

- $a^{(1)} = x$



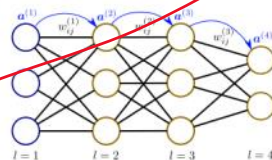
11

Backpropagation algorithm

Suppose we have a MLP and one training example (x, y) .

Step 1: We first perform a **forward propagation pass**:

- $a^{(1)} = x$
- $z^{(2)} = w^{(1)} a^{(1)}$



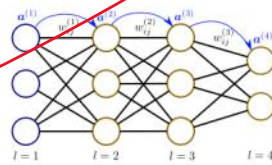
11

Backpropagation algorithm

Suppose we have a MLP and one training example (x, y) .

Step 1: We first perform a **forward propagation pass**:

- $a^{(1)} = x$
- $z^{(2)} = w^{(1)} a^{(1)}$
- $a^{(2)} = g(z^{(2)})$



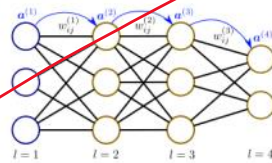
11

Backpropagation algorithm

Suppose we have a MLP and one training example (x, y) .

Step 1: We first perform a **forward propagation pass**:

- $a^{(1)} = x$
- $z^{(2)} = w^{(1)} a^{(1)}$
- $a^{(2)} = g(z^{(2)})$
- $z^{(3)} = w^{(2)} a^{(2)}$



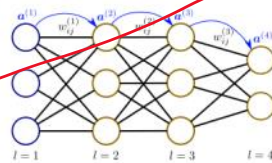
11

Backpropagation algorithm

Suppose we have a MLP and one training example (x, y) .

Step 1: We first perform a **forward propagation pass**:

- $a^{(1)} = x$
- $z^{(2)} = w^{(1)} a^{(1)}$
- $a^{(2)} = g(z^{(2)})$
- $z^{(3)} = w^{(2)} a^{(2)}$
- $a^{(3)} = g(z^{(3)})$



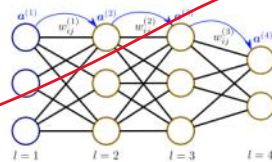
11

Backpropagation algorithm

Suppose we have a MLP and one training example (x, y) .

Step 1: We first perform a **forward propagation pass**:

- $a^{(1)} = x$
- $z^{(2)} = w^{(1)} a^{(1)}$
- $a^{(2)} = g(z^{(2)})$
- $z^{(3)} = w^{(2)} a^{(2)}$
- $a^{(3)} = g(z^{(3)})$
- $z^{(4)} = w^{(3)} a^{(3)}$



11

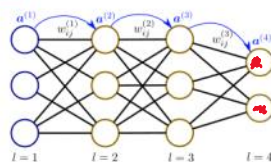
↓
exemplo

Backpropagation algorithm

Suppose we have a MLP and one training example (x, y) .

Step 1: We first perform a **forward propagation pass**:

- $a^{(1)} = x$
- $z^{(2)} = w^{(1)} a^{(1)}$
- $a^{(2)} = g(z^{(2)})$
- $z^{(3)} = w^{(2)} a^{(2)}$
- $a^{(3)} = g(z^{(3)})$
- $z^{(4)} = w^{(3)} a^{(3)}$
- Output $a^{(4)} = g(z^{(4)})$



At this step we know the output of the current MLP setup.

11

Backpropagation algorithm

2. evaluate for each node the error $\delta_j^{(k)}$ for $k = 2, 3, \dots, L$.

Some remarks:

It is possible to proof using derivative chain rules that:

$$\nabla_{ij}^{(l)} J = \frac{\partial J}{\partial z_i^{(l+1)}} a_j^{(l)} \equiv \delta_i^{(l+1)} a_j^{(l)},$$

for $l = 1, \dots, L-1$.

12

Backpropagation algorithm

2. evaluate for each node the error $\delta_j^{(k)}$ for $k = 2, 3, \dots, L$.

Some remarks:

It is possible to proof using derivative chain rules that:

$$\nabla_{ij}^{(l)} J = \frac{\partial J}{\partial z_i^{(l+1)}} a_j^{(l)} \equiv \delta_i^{(l+1)} a_j^{(l)},$$

for $l = 1, \dots, L-1$.

The recursive relation for the error is:

$$\delta_i^{(l)} = \sum_k w_{ki}^{(l)} \delta_k^{(l+1)} \cdot g'(z_i^{(l)})$$

and at $l = L$, i.e. the highest l index:

$$\delta_i^{(L)} = \frac{\partial J}{\partial a_i^{(L)}} \cdot g'(z_i^{(L)})$$

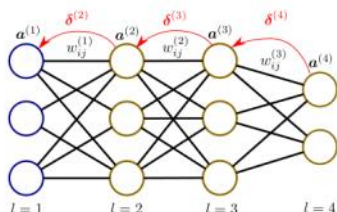
where $g'(z_i^{(l)}) = a_i^{(l)}(1 - a_i^{(l)})$ if g is the sigmoid function.

12

Backpropagation algorithm

Example: evaluating error $\delta_j^{(l)}$ for a MLP with sigmoids in the hidden layers and linear activation function in the output layer:

- $\delta^{(4)} = a^{(4)} - y$ ← errore output layer, scelta
- $\delta^{(3)} = (w^{(3)})^T \delta^{(4)} \cdot (a^{(3)}(1 - a^{(3)}))$
- $\delta^{(2)} = (w^{(2)})^T \delta^{(3)} \cdot (a^{(2)}(1 - a^{(2)}))$



13

Backpropagation algorithm summary

Data: training set $(x^{(i)}, y^{(i)})$ with $i = 1, \dots, m$ examples.

Result: the trained neural network

Initialize network weights;

while stopping criterion is not satisfied do

Set all $\Delta w_{ij}^{(l)} = 0$.

for $k = 1$ to m do

Perform forward pass and compute $a^{(l)}$ for $l = 1, 2, 3, \dots, L$;

Perform backward pass and compute $\delta^{(l)}$ for $l = 2, \dots, L$;

$\Delta w_{ij}^{(l)} := \Delta w_{ij}^{(l)} + a_j^l \delta_i^{(l+1)}$

end

Update network weights using gradient descent;

end

backpropagation

← gradient descent

14

Training neural networks

Some remarks and example of neural network initialization:

- **zero**: all weights are set to zero so all neurons perform the same calculation. The complexity of the neural network is equivalent to a single neuron.
- **random**: breaks parameter symmetry.
- **glorot/xavier**: initialize each weight with a small Gaussian value with mean zero and variance based on the in/out size of the weight.
- **he**: avoid activation function saturation. Weights are random initialized considering the size of the previous layer.

← un Hyperparametro

← evita il problem vanishing effect

Se inizialmente sono tutto a 0, ho difficoltà a muovermi, l'ottimizzatore non sa dove andare.

Uso quindi il random che rompe la simmetria, o tecniche più sofisticate come

15

Neural networks zoo

Artificial neural networks architectures



Lo cellule diverse, l'informazione ritorna

Se ho timeseries avere effetti di memoria può dare benefici(RNN,LSTM)

AE, prende dati, li comprime, e li riallarga. Tipico per comprimere informazione. Ad esempio compressione immagini con molti pixel zero.

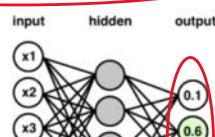
AutoEncoder quindi porta i dati in un sottospazio, e la seconda parte lo posso pensare come un decoder.

Deep convolutional layer, usati per leggere immagini, voglio quindi un'architettura per comprimere l'immagine(in modo diverso da AE). Per fare un classificare metto poi un multilayer perceptron.

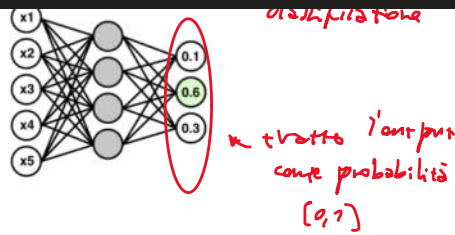
Attention Network, posso avere anche che l'informazione sia lavorata in diverse architetture in modo indipendente. Per esempio combino la classe e la posizione

MLP

- Sequential model (MLP): regression and classification



voglio fare classificazione



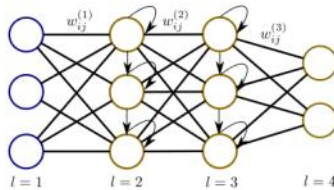
→ Attention network, posso avere anche che l'informazione sia lavorata in diverse architetture in modo indipendente. Per esempio combinare la classe e la posizione dell'oggetto

softmax normalizza l'output

Artificial neural networks architectures

Some examples of neural network popular architectures:

- **Recurrent neural networks:** neural networks where connections between nodes form a directed cycle.
 - built-in internal state memory
 - built-in notion of time ordering for a time sequence

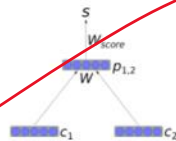


In modelli ricorrenti ho connessioni cicliche.

Utile se ho un alto indice di correlazione

Artificial neural networks architectures

- **Recursive neural networks:** a variation of recurrent neural network where pairs of layers or nodes are merged recursively.
 - successful applications on natural language processing.
 - some recent applications for model inference.

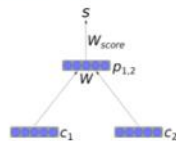


Recursive, variazione di quelle ricorrenti, l'informazione viene divisa in coppie di layers, che vengono poi uniti? Utile prima dei transformers.

Usato per linguaggio

Artificial neural networks architectures

- **Recursive neural networks:** a variation of recurrent neural network where pairs of layers or nodes are merged recursively.
 - successful applications on natural language processing.
 - some recent applications for model inference.



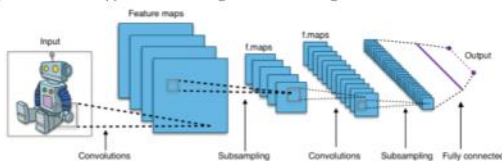
- **Long short-term memory:** another variation of recurrent neural networks composed by custom units cells:
 - LSTM cells have an input gate, an output gate and a forget gate.
 - powerful when making predictions based on time series data.

Activation molto complessa, variazione di un RNN, e setto quanta informazione tenere.

Utile per serie temporali

Artificial neural networks architectures

- **Convolutional neural networks:** multilayer perceptron designed to require minimal preprocessing, i.e. space invariant architecture.
 - the hidden layers consist of convolutional layers, pooling layer, fully connected layers and normalization layers
 - great successful applications in image and video recognition.

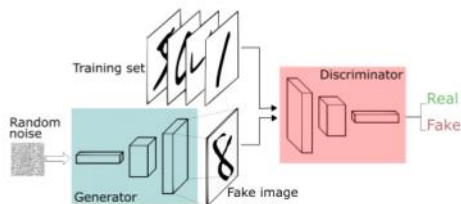


Nelle CNN, per evitare di passare tutti i pixel, uso piccole compressioni, che con prodotti di convoluzione crea feature maps(convoluzione tra kernel e pixel vicini), che poi sopprimo con subsampling.

In questo modo tolgo la ridondanza, fino ad arrivare in spazi più piccoli su cui usare un percettrone

Artificial neural networks architectures

- **Generative adversarial network:** unsupervised machine learning system of two neural networks contesting with each other.
 - one network generate candidates while the other discriminates.



21

Digital images treatment

Pattern recognition algorithms:

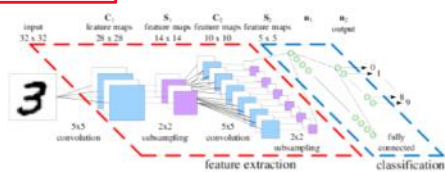
- **semantic segmentation:** predict labels for pixels
- **classification + localization:** predict object and its location
- **object detection:** identify multiple objects and locations
- **instance segmentation:** identify multiple objects and masks



22

Models for digital image treatment

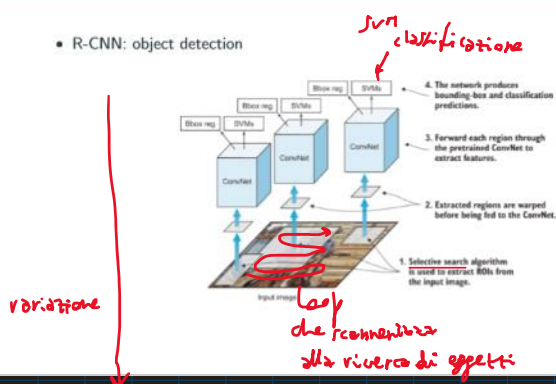
- **CNN: image classification**



23

Models for digital image treatment

- **R-CNN: object detection**



24

Modelli generativi di tipo avversario. In pratica ho 2 architetture, generatore e discriminatore.

Il discriminatore deve dire se l'immagine è vera o falsa.

Il generatore invece vuole generare copie di immagini, a partire da del rumore.

1 step: prendo random noise e genererò cosa casuale. Il discriminatore indovinerà facilmente
2 step: alleno poi il generatore, e itero
3 step: alla fine si allenano a vicenda

Posso fare tanti processi

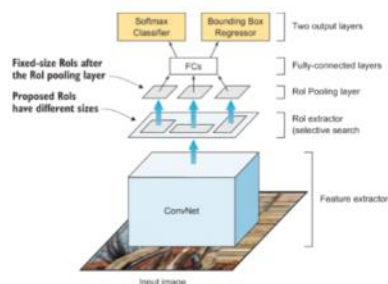
1v4: 4 oltre a definire i bordi classifica anche, con una sola architettura

Qua stiamo assumendo ci sia un numero

Qua capiamo anche se c'è un numero

Models for digital image treatment

- Fast R-CNN: object detection

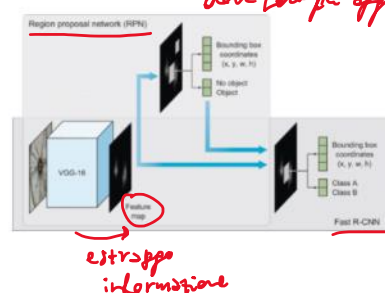


volgarizzazione

25

Models for digital image treatment

- Faster R-CNN: object detection



dove sono gli oggetti!

estrage informazioni

quello prima

26

Models for digital image treatment

Transfer learning: a research problem in machine learning (ML) that focuses on storing knowledge gained while solving one problem and applying it to a different but related problem.



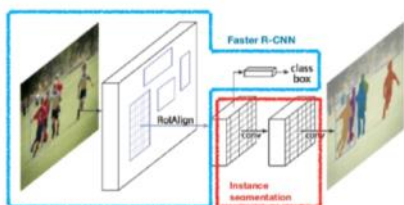
27

Prendo una CNN allenata su un dataset diverso dal mio. Quando il modello è potente e flessibile, lo prendo e lo riapplico al mio dataset molto più piccolo.

Se lascio solo gli ultimi layers liberi di essere allenati, questo funziona molto bene. Le CNN all'inizio estraggono informazioni in modo abbastanza buono

Models for digital image treatment

- Mask-RCNN or UNet: instance segmentation



Ultimo modello, dove sono i pixel di oggetti? combinazione tra fast R-CNN e colorazione così. Altro esempio di modello composto, per esempio un sottomodello qua si occupa della colorazione di pixel

28

Transformer usa rnn

Deep Learning - lecture 4

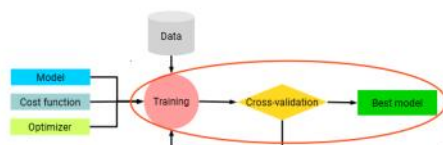
Model selection and hyperparameter tune

Prof. Stefano Carrazza

University of Milan and INFN Milan

Introduction

Outline



Concentramoci sul lato test

Hyperparameters summary

So far we have encountered the following problems:

- **Model:**
 - model architecture / size
 - if NN: layers, nodes, activation functions
 - regularization techniques including early stopping techniques, weight decay, etc.
- **Training:**
 - performance metrics
 - optimizer configuration, e.g.: η , scheme, etc.
 - cross-validation split fractions
- **Dataset:**
 - size (gather more data?)
 - unbalanced data

Hyperparameters summary

So far we have encountered the following problems:

- **Model:**
 - model architecture / size
 - if NN: layers, nodes, activation functions
 - regularization techniques including early stopping techniques, weight decay, etc.
- **Training:**
 - performance metrics
 - optimizer configuration, e.g.: η , scheme, etc.
 - cross-validation split fractions
- **Dataset:**
 - size (gather more data?)
 - unbalanced data

Each choice should be tested $\rightarrow$ large space $\rightarrow$ difficult / time consuming.

2

I parametri in gioco sono questi.

Alcuni di questi sono scelte che faccio a priori, e chiamo quindi hyperparametri (li scelgo a priori)

Ho quindi 2 livelli di allenamento

Practical methodology

Don't get lost!

Designing a practical pipeline process:

- Estimate current **state-of-the-art performance**.
- Define **realistic project goals**, simplify / accelerate algorithms.
- Propose initial **performance metrics** matching the project goals.
- Perform **incremental changes** iteratively (data, hyperparameter, algorithms, etc.).

3

Cosa devo fare quando ho bisogno di un modello?

1. Studiare se esistono modelli già pronti, che risolvono il problema. Devo quindi capire i limiti a cui la letteratura è arrivata.
2. Determino ora cosa voglio fare, migliorare fit?
3. Proporre un setup di metriche ragionevoli, in modo da poter fare modifiche incremental...

Example 1: auto-tuning model's capacity

How to simplify model capacity selection? **Early stopping techniques**



Monitor the cost function for the validation set and stop when it stops improving:

- look at the variation in a moving window
- stop at the minimum of the validation set (lookback method).

4

auto tuning

Questo è un esempio. Mi fermo quando la validation è minima e dico che è la migliore.

Qua non ho modificato l'architettura, ma ho determinato quando fermarmi. Posso poi usare una moving window per calcolare le medie e capire quando fermarmi.

Ma non cambio l'architettura, ma dico che è sufficiente per rappresentare i dati, e devo solo capire quando fermarmi

Example 2: auto-tuning model's capacity

How to simplify model capacity selection? **Neural Network Dropout**

At each training stage:

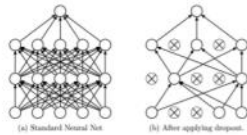
- individual nodes and related incoming and outgoing edges are dropped-out of the neural network with a fixed probability.
- the reduced NN is trained on the data.
- the removed nodes are reinserted in the NN with their original weights.



Se uso questo per fermare il modello, potrei in qualche modo spostando dell'informazione della validation dentro il modello. Stiamo usando la feature del modello per capire quando fermarlo.

Il test set deve quindi sempre essere **ESTERNO**, e mai usato per determinare parametri (come per esempio qua). Triplo split (train-validation)-test

Un'altra possibilità è usare un **dropout**, layer che controllerà in modo stocastico quali nodi possono o no collegarsi tra loro. In questo modo parto da architetture



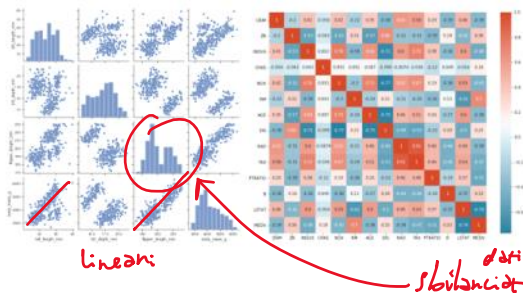
5

Un'altra possibilità è usare un **dropout**, layer che controllerà in modo stocastico quali nodi possono o no collegarsi tra loro. In questo modo parto da architetture troppo flessibili, in architetture più compatte.

Data considerations

Data considerations

Step 1: Understand your data, extract correlations, perform minimal feature extraction.



6

La prima cosa da fare è prendere il dataset e analizzarlo, per capire come sono fatti. Qua sto cercando correlazioni.

Nel caso di linearità posso escludere dei valori. Se ho tanti parametri conviene vedere una matrice di correlazione. E prendo poi i parametri logicamente sensati, e non dare tutti i parametri all'architettura.

Questo aiuta anche a capire se ho dati sbilanciati.

Data considerations

Gathering more data is usually crucial but before:

Step 2:

- check the performance with the current training set, if its performance is:
 - ✗ **poor** → increase model size and tune the optimizer.
 - ✓ **acceptable** → check test set performance. → *controlla il test*

Step 3:

- if tuned models **fail** → check data for noise or inconsistencies, collect new data
- if test set performance is **poor** → gather more data if possible
 - if not possible reduce the size of the model or improve hyperparameter tuning.

Step 4:

- estimate how much additional training data is needed.
- if gathering much more data is **not feasible** → improve the learning algorithm itself.

7

Guida.

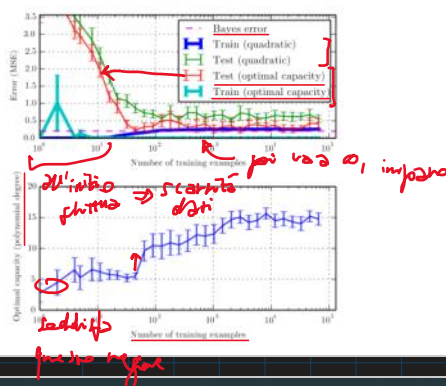
- Analisi dati
- Provo a fare un fit. Se il modello funziona bene. Se non funziona: modifico il modello, l'ottimizzatore, gli iperparametri. Questo può essere fatto a mano, ma per cose più avanzate posso usare tecniche più avanzate.
- Se il fine tuning fallisce, posso magari controllare i dati, o provare ad espanderli.

Se ottengo un buon train, ma il test non è buono potrei avere bisogno di più dati (ho imparato le risposte senza generalizzare).

Se non posso aggiungere dati, posso ridurre i parametri

- Altrimenti, stimo quanti dati potrei avere bisogno

Data considerations



8

Lavoro un po' empirico

N.B. nel caso di quadratico, per pochi dati è buono, poi con più dati peggiora. Il test in modo analogo all'inizio va male, e poi si abbassa.

Il problema è determinar la soglia in funzione del dataset

A un certo punto non miglio, vado in overlearning

Unbalanced data

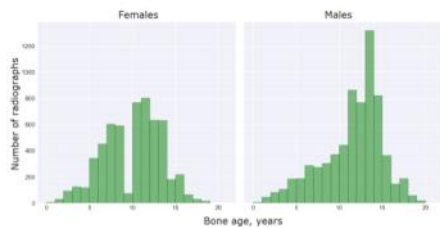
Unbalanced datasets are common issues:

Females Males

Età assai - età anagrafica?

Unbalanced data

Unbalanced datasets are common issues:



Solution → perform class weighting, oversampling, data augmentation.

9

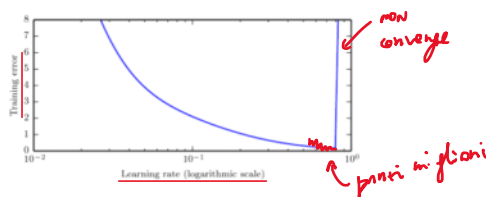
Hyperparameter tune

Manual hyperparameter tuning

Manual approach goal → achieve good performance on the test set.

Some examples of effect of hyperparameters on model capacity:

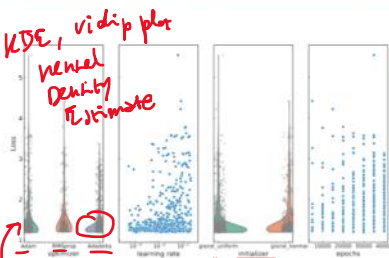
- Number of layers/nodes → increases capacity when **increased**.
- Learning rate → increases capacity when **tuned optimally**.
- Weight decay → increases capacity when **decreased**.
- Dropout rate → increases capacity when **decreased**.



10

Automatic hyperparameter optimization algorithms

Hyperparameter tuning is an optimization problem thus we can automate the process.



Common approaches:

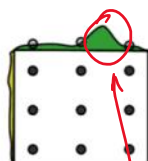
- grid search
- random search
- bayesian optimization
- gradient-based optimization
- evolutionary optimization

11

Grid search

Grid search: searching through a manually subset range of the hyperparameter space.

- Train model for every grid point of the hyperparameter space.
- Allocate initial grids following a logarithmic scale, perform zoom in another round.
- Monitor the best validation set error → best hyperparameter values.



tutti i tentativi

The disadvantage: with n values and m parameters the number of trials is $O(n^m)$

vantaggio: fit completo

Età ossea = età anagrafica?

Il dataset fa vedere da neonati a 20 anni. Il bias è che i problemi si notano in fase adolescenziale.

Se dovessi fare un classificatore che dalla foto e sesso, sarà bravo dire se è attorno il picco. Se invece mi sposto fuori ho pochi campioni.

Potrei far sì che il dataset abbia stesso numero di cose, posso poi modificare il peso nella loss function, o posso fare oversampling, ossia l'algoritmo di training genera con frequenza più alta ai "lati", nei punti meno frequenti, o ancora con data augmentation, posso prendere immagini e scalarle, rotarle, o modificarle un po' creando dati artificiali ma con caratteristiche diverse.

Questo qualora la raccolta di dati non serve a ribilanciare il dataset

Parliamo ora come tunare gli iperparametri.

Un primo approccio è quello di manualmente provare un po' a caso.

Se aumento il numero di parametri avrò più capacità.

Learning rate troppo piccolo impara lento e troppo grande impara male

Questo va bene con pochi parametri

Se ho tanti parametri, questo mostra 1k tentativi per diverse tipologie di hyperparametri

1000 ← più tentativi che cadono qui

quale prendo?

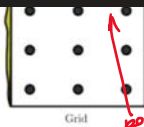
Se prendo il punto migliore, in linea di principio va bene, ma non voglio il setup migliore in 1000 fit, ma voglio cose più stabili. Quindi in pratica non guardo il punto migliore, ma faccio un bilanciamento.

Quindi voglio le scelte che mi danno code piccole.

Sono quindi decisioni che posso automatizzare, ma voglio avere sotto controllo.

Avere una loss 5% migliore non cambia molto, ma avere un comportamento più stabile sì. Sono tutti minimi locali praticamente.

Come faccio questi plot?



The disadvantage: with n values and m parameters the number of trials is $\mathcal{O}(n^m)$

vantaggio: fit completo
 posso avere punti in posizioni non ideali e perdo il picco
 posso parallelizzarlo

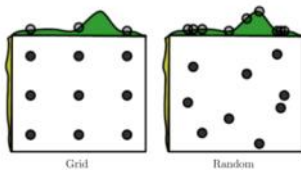
praticamente.

Come faccio questi plot?

Random search

Random search: sample trial points from a marginal distribution for each hyperparameter.

- Do not discretize or bin the values of the hyperparameters.
- The marginal distribution will perform independent explorations of hyperparameters.



Cerco in modo casuale, potrebbe capitare una situazione in cui trovo punti più interessanti.

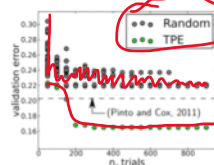
2 tecniche semplici, ma che hanno bisogno di molti calcoli

Model-based hyperparameter optimization

Idea:

- Perform a training using a set of hyperparameters.
- Define the cost function to be optimized as the validation set error.
- Use sequential model-based optimization (SMBO) approach, or algorithms which monitors the numerical gradient from the loss function.

Example: Tree-structured Parzen Estimator (TPE)



nel migliore più veloce

Sebbene non hanno loss con forme analitiche, e non posso fare gradienti. Ma se riesco a ricostruire la funzione, e ne capisco il comportamento. Se ne creo una funzione di probabilità

SMBO

SMBO minimizes functions $f: X \rightarrow \mathcal{R}$ where each evaluation is very expensive.

The f function is replaced by a surrogate function, $\tilde{f}$, easier to manage.

The surrogate function proposes a new search point x_{i+1} , $f(x_{i+1})$ is computed and $\tilde{f}$ updated or recomputed to approximate better the true loss function.

```

Data: loss function  $f$ , initial surrogate  $\tilde{f}_0$ , number of trials  $T$ 
Result: Candidate  $x_{best}$  for the minimum of  $f$ 
Set trials history  $H = \emptyset$ ;
for  $i = 1$  to  $T$  do
   $x^* \leftarrow \text{argmin}_x L(x, \tilde{f}_{i-1})$ ;
  Compute  $f(x^*)$ ;
   $H \leftarrow H \cup \{x^*, f(x^*)\}$ ;
  Model a new surrogate function  $\tilde{f}_i$  using  $H$ ;
end
  
```

Where $L(x, \tilde{f})$, the criterion, and $\tilde{f}$ depend on the specific algorithm.

funziona con

La più popolare è la tpe (tree-structured parzen estimator?) per la surrogate function

$\tilde{f}$ stima la densità

Voglio stimare la densità con poche chiamate

TPE

The Tree-structured Parzen Estimator (TPE) algorithm is a SMBO where the surrogate model is a probabilistic model $p(y|x)$, which chooses the next trial point by optimizing the Expected Improvement criterion:

$$EI_{y^*}(x) = \int_{-\infty}^{\infty} \max(y^* - y, 0) p(y|x) dy$$

which measures how much the loss function is expected to be lower than a threshold value y^* , chosen so that $p(y < y^*) = \gamma$ where γ is a parameter of the algorithm.

Probabilità di estrazione di un hyperparametro in funzione dei risultati di un fit precedente. Uso qua la tpe quindi la distanza dal fit.

Mediamente è robusto, e si dimostra empiricamente che spesso riduce i tentativi.

Questo modello può fallire per il problema di trovare la probabilità $p(y|x)$. Per non entrare in regioni dello spazio con poche soluzioni, inizio con una random search negli hyperparametri, e poi con quella eseguo la tpe, che esegue la ricerca in modo più tranquillo.

TPE

The $p(y|x)$ is computed via Bayes' theorem through $p(x|y)$

TPE

The $p(y|x)$ is computed via Bayes' theorem through $p(x|y)$

$$p(x|y) = l(x) \quad \text{if } y < y^*; \quad g(x) \quad \text{if } y \geq y^*$$

where $l(x)$ and $g(x)$ are probability distributions estimated by using the trials x_i such that $f(x_i)$ is respectively lower and higher or equal y^* .

The Expected Improvement for the TPE admits a closed form solution:

$$EI_{y^*}(x) = \int_{-\infty}^{\infty} \max(y^* - y) \frac{p(x|y)p(y)}{p(x)} dy \propto \left(\gamma + \frac{g(x)}{l(x)} (1 - \gamma) \right)^{-1}$$

Th Bayes

17

Code libraries and algorithms



originale

pruning (togliere dati)

- scikit-learn: grid and random search.
- Hyperopt: grid, random and TPE.
- Optuna: grid, random, TPE, CMAES.
- Ray Tune: grid, random, bandit, blended, cost-frugal, TPE, gradient-free, etc.

genetico

funziona più in generale

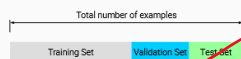
più elitico

18

Cross-validation

Cross-validation

The hyperparameter tune procedure still requires the training/validation/test split to choose for the best model.



Problems:

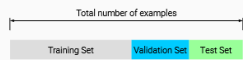
- how to perform the data split when the available data set is small?
- how to define a suitable split?

19

hyperparametri, e poi con quella eseguo la tpe, che esegue la ricerca in modo più tranquillo.

Cross-validation

The hyperparameter tune procedure still requires the training/validation/test split to choose for the best model.



Problems:

- how to perform the data split when the available data set is small?
- how to define a suitable split?

Solution:

Use **cross-validation** algorithms to access the quality of your model + hyperparameter choice.

19

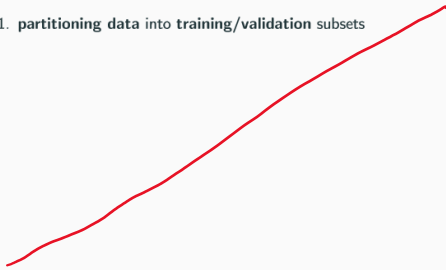
Non posso usare la validation per stimare il modello, quindi ho bisogno anche del test. Per fare questo split ho bisogno di tanti dati.

Per risolvere parzialmente la cosa ho tecniche di cross validation

Cross-validation

Cross-validation performs a rotation estimation by:

1. partitioning data into training/validation subsets

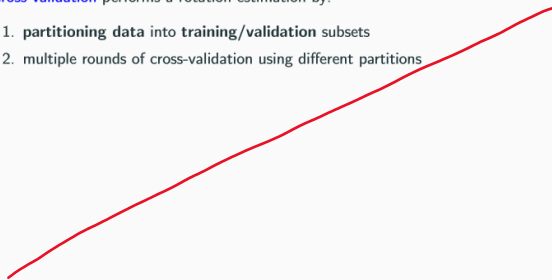


20

Cross-validation

Cross-validation performs a rotation estimation by:

1. partitioning data into training/validation subsets
2. multiple rounds of cross-validation using different partitions

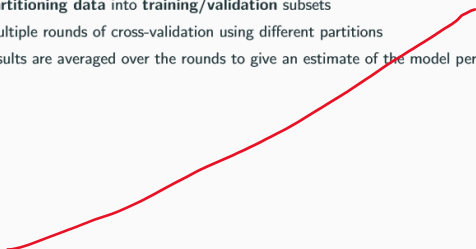


20

Cross-validation

Cross-validation performs a rotation estimation by:

1. partitioning data into training/validation subsets
2. multiple rounds of cross-validation using different partitions
3. results are averaged over the rounds to give an estimate of the model performance



20

Cross-validation

Cross-validation performs a rotation estimation by:

1. partitioning data into training/validation subsets
2. multiple rounds of cross-validation using different partitions
3. results are averaged over the rounds to give an estimate of the model performance

Common approaches to cross-validation:

- **Exhaustive cross-validation:** test all possible ways to divide the original sample into a training and a validation set.

20

Cross-validation

Cross-validation performs a rotation estimation by:

1. partitioning data into training/validation subsets
2. multiple rounds of cross-validation using different partitions
3. results are averaged over the rounds to give an estimate of the model performance

Common approaches to cross-validation:

- **Exhaustive cross-validation:** test all possible ways to divide the original sample into a training and a validation set.
 - Leave- p -out: uses p observations as validation set.

20

Cross-validation

Cross-validation performs a rotation estimation by:

1. partitioning data into training/validation subsets
2. multiple rounds of cross-validation using different partitions
3. results are averaged over the rounds to give an estimate of the model performance

Common approaches to cross-validation:

- **Exhaustive cross-validation:** test all possible ways to divide the original sample into a training and a validation set.
 - Leave- p -out: uses p observations as validation set.
 - Leave-one-out: set $p = 1$.

20

Cross-validation

Cross-validation performs a rotation estimation by:

1. partitioning data into training/validation subsets
2. multiple rounds of cross-validation using different partitions
3. results are averaged over the rounds to give an estimate of the model performance

Common approaches to cross-validation:

- **Exhaustive cross-validation:** test all possible ways to divide the original sample into a training and a validation set.
 - Leave- p -out: uses p observations as validation set.
 - Leave-one-out: set $p = 1$.
- **Non-exhaustive cross-validation:** do not test all possible ways to divide the original sample but use discrete subsamples.

20

Cross-validation

Cross-validation performs a rotation estimation by:

1. partitioning data into training/validation subsets
2. multiple rounds of cross-validation using different partitions
3. results are averaged over the rounds to give an estimate of the model performance

Common approaches to cross-validation:

- **Exhaustive cross-validation:** test all possible ways to divide the original sample into a training and a validation set.
 - Leave- p -out: uses p observations as validation set.
 - Leave-one-out: set $p = 1$.
- **Non-exhaustive cross-validation:** do not test all possible ways to divide the original sample but use discrete subsamples.
 - k -fold cross-validation.

20

Per esempio posso ripartirli.

Per esempio posso tabulare i risultati

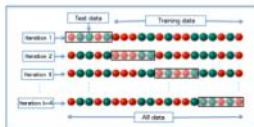
Posso usare batch. Una volta deciso la regola di partizione. Punti 1 ... 3?

Example **k-fold cross-validation**

k -fold cross-validation:

1. the original data is randomly partitioned into k equal sized subsamples.
2. from the k subsamples, a single subsample is used as validation data and the remaining $k - 1$ subsamples are used as training data.
3. repeat the process k times by changing the validation and training partitions.
4. compute the average over the k results.

Example of k -fold with $k = 4$:



21

Nel caso della kfold dico quante partizioni mi interessano, per esempio $k=4$, quindi devo almeno fare 4 fit, in cui modifico il contenuto delle partizioni.

N.B. qua c'è scritto test, voglio validare il modello senza usare i dati per capire quando fermarmi, poi faccio la media sui vari fit

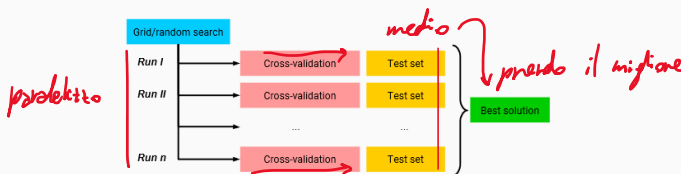
Un'altra caratteristica importante è che se è tutto molto stabile, la dispersione della error function è bassa, posso trovare una configurazione di dati, togliere la kfold, e fittare tutti i dati. Questo posso farlo se ho pochi dati per determinare tutto quello che mi serve, hyperparametri e tutto. Ma devo essere sicuro che la stabilità dei kfold sia molto stabile.

Quello sarà il best fit.

Se invece ho molti dati bona

Complete recipe

Perform hyperparameter tune coupled to cross-validation:



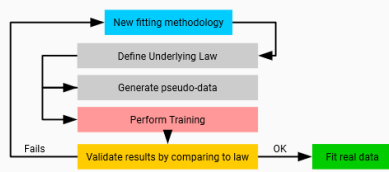
Easy parallelization at search and cross-validation stages.

22

Closure testing

Closure tests

Validation and optimization of fitting strategy performed on closure test with known underlying law.



23

Può essere una tecnica utile.

Fissata parametri e architetture, genero dati falsi, e li uso per fittare.

Ci permette di capire quanto la metodologia è in grado di adattarsi ai dati.

Se riesco a rigenerare dati simili è stabile

Si chiama di closure perché da una metodologia creo dati artificiali, e se ritrovo i dati originali la metodologia si è adattata ai dati.

Può essere utile se ho un modello che mi sa generare dati, anche solo per stimare errori di modelli teorici. Posso aggiungere incertezze e vedere cosa succede

Deep Learning - lecture 5

Recurrent Neural Networks

Prof. Stefano Carrazza

University of Milan and INFN Milan

Computational Graphs

Introducing computational graphs

Let's suppose to compute the following expression:

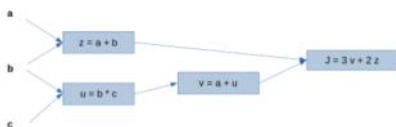
$$J(a, b, c) = 3(a + bc) + 2(b + a)$$

1

Introducing computational graphs

Let's suppose to compute the following expression:

$$J(a, b, c) = 3(a + bc) + 2(b + a)$$



The evaluation of z and u can be performed in parallel.

1

Posso convertire questa funzione in un grafico, individuando le operazioni indipendenti tra loro.

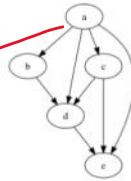
Questo mi permette di capire cosa posso parallelizzare.

Questo è quello che avviene in tensorflow

Introducing computational graphs

From a technical point of view a **computational graph**:

- formalizes the structure of a **set of computations**
- defines a **chain of events**
- it represents the basic rules for NNs

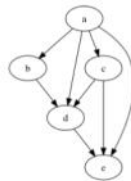


2

Introducing computational graphs

From a technical point of view a **computational graph**:

- formalizes the structure of a **set of computations**
- defines a **chain of events**
- it represents the basic rules for NNs



A graph is defined by a set of vertices (nodes) and edges (links).

A **direct graph** contains edge orientations and may contain **loops** (e.g. recurrency).

An **acyclic direct graph (DAG)** is a directed graph with no directed cycles.

2

Posso poi fare questo in modo più tecnico:
Grafici computazionali

Qua va tutto in cascata, directed graph, senza cicli

dash library
↳ gestione grafici
↓
parallelizza

Introducing computational graphs

For example, given a system represented by the graph:



we can determine the state $s^{(t)}$ at any position t with:

$$s^{(t)} = f(s^{(t-1)}; w)$$

which can be **unfolded** (not involving recurrence):

$$s^{(4)} = f(s^{(3)}; w) = f(f(f(s^{(1)}; w); w); w)$$

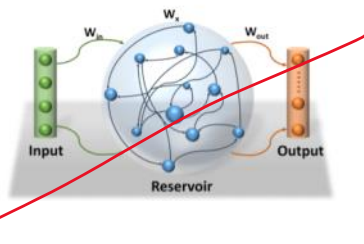
folded

3

Recurrent neural networks

Recurrent neural networks

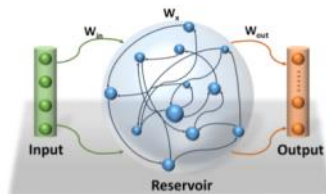
Let's consider an **Echo State Network**:



4

Recurrent neural networks

Let's consider an **Echo State Network**:



The behaviour is non-linear, the only weights that are modified during training are for the synapses that connect the hidden neurons to output neurons.

4

Echo state network: rete rnn migliore(?)

Ho diversi nodi, con connessioni qualunque

Questa architettura è molto espressiva perché può essere un po' tutto, ma la convergenza del gradiente, e la backpropagation è abbastanza complicata.

Essendo che è però "completamente" non lineare ho espressività massima(molto buona)

Recurrent neural networks

Suppose to operate on a sequence $x^{(t)}$:

$$(x^{(t)}, x^{(t-1)}, x^{(t-2)}, \dots, x^{(2)}, x^{(1)})$$

How can we take advantage of the **sequence correlation / causality**?

5

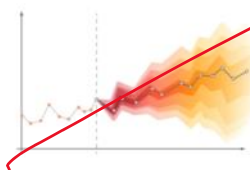
Recurrent neural networks

Suppose to operate on a sequence $x^{(t)}$:

$$(x^{(t)}, x^{(t-1)}, x^{(t-2)}, \dots, x^{(2)}, x^{(1)})$$

How can we take advantage of the **sequence correlation / causality**?

Application examples:



- Time series predictions
- Speech/Handwriting recognition
- Text translation
- Anomaly detection
- Protein homology detection
- ...

5

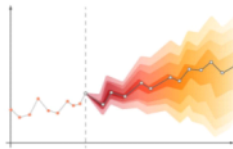
Recurrent neural networks

Suppose to operate on a sequence $x^{(t)}$:

$$(x^{(t)}, x^{(t-1)}, x^{(t-2)}, \dots, x^{(2)}, x^{(1)})$$

How can we take advantage of the **sequence correlation / causality**?

Application examples:



- Time series predictions
- Speech/Handwriting recognition
- Text translation
- Anomaly detection
- Protein homology detection
- ...

⇒ Recurrent Neural Networks (RNNs), Rumelhart et al. 1986

paper originale

Consideriamo una sequenza di elementi, anche di vettori. Siccome si tratta di misure, in linea di principio indipendenti, come posso preservare correlazione e causalità? Questo si riconduce per esempio a serie temporali, o frasi in cui l'ordine è importante.

Le reti base non sono fatte per ricordarsi delle correlazioni, ma sono "oneshot".

Posso quindi cercare una soluzione con le RNN

Recurrent neural networks

Suppose to operate on a sequence of vectors $x^{(t)}$:

$$(x^{(t)}, x^{(t-1)}, x^{(t-2)}, \dots, x^{(2)}, x^{(1)})$$

A simple example of recurrent network with no outputs is defined by:

$$h^{(t)} = f(h^{(t-1)}, x^{(t)}; w)$$

Recurrent neural networks

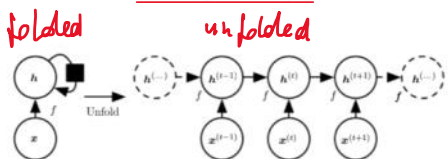
Suppose to operate on a sequence of vectors $x^{(t)}$:

$$(x^{(t)}, x^{(t-1)}, x^{(t-2)}, \dots, x^{(2)}, x^{(1)})$$

A simple example of recurrent network with no outputs is defined by:

$$h^{(t)} = f(h^{(t-1)}, x^{(t)}; w)$$

Graphically:



The input x is incorporated into the state h which is passed forward through time.

Ingrediente fondamentale

La x viene incorporata nello stato h , e viene passato step by step

Ponte tra tempo passato e futuro per preservare l'informazione. h contiene già informazione non lineare salvata

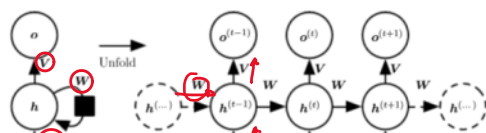
Recurrent neural networks

From a practical perspective, there are three RNN important design patterns:

Recurrent neural networks

From a practical perspective, there are three RNN important design patterns:

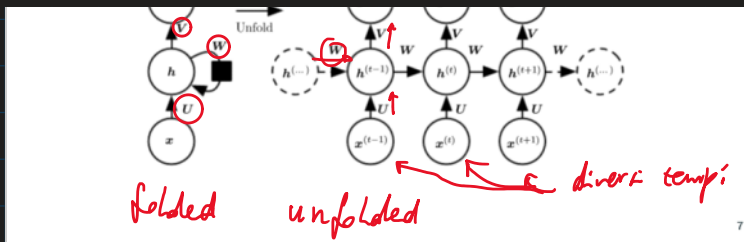
1. RNN produces an output at each time step and have recurrent connections for hidden units.



architettura base

Posso usare questo in delle reti

U, W e V matrici di pesi

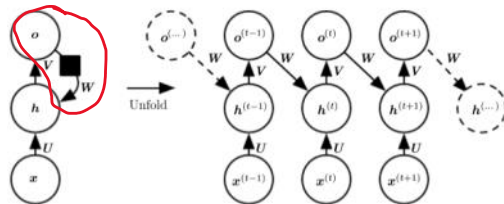


U, W e V matrici di pesi

Dati con correlazioni temporali, le informazioni vengono trasmesse tra i vari tempi

Recurrent neural networks

2. RNN produces an output at each time step and have recurrent connections only from the output at one time step to the hidden units at the next time step.



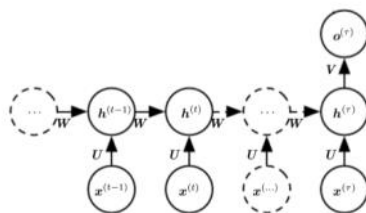
Non ho più una connessione ricorrente sincrona, ma inizio partendo dal primo step. Ho quindi un delay tra l'informazione di output e quelle future.

Il loop interno è quindi spostato, usando direttamente l'output delle operazioni fatte, che viene poi modulato.

Lo posso fare perché quelli di output sono magari quelli che contengono più informazione, e voglio preservarla

Recurrent neural networks

3. RNN with recurrent connections between hidden units, that read an entire sequence and then produce a single output.

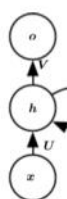


Un terzo caso e se ho ricorrenza tra le h, e poi predico un unico valore futuro. Simile al primo esempio poiché non crea output, ma ne crea giusto uno alla fine

3 tipologie

Example of RNN forward propagation

Let's consider design 1, i.e. RNN produces an output at each time step and have recurrent connections for hidden units.



Forward propagation:

Assuming tanh activation function:

$$\begin{aligned} a^{(t)} &= b + W h^{(t-1)} + U x^{(t)} \\ h^{(t)} &= \tanh(a^{(t)}) \\ o^{(t)} &= c + V h^{(t)} \end{aligned}$$

output

where b and c are the bias vectors, and:

- U input-to-hidden connections
- V hidden-to-output connections
- W hidden-to-hidden connections

esempio caso 1

Per capire quale scegliere, devo riflettere sul mio problema, e dove l'informazione si trova

Example of RNN gradient evaluation

RNN gradients are evaluated using back-propagation through time (BPTT).

Considering a loss function where $\{y^{(1)}, \dots, y^{(r)}\}$ are targets:

$$J(\{x^{(1)}, \dots, x^{(r)}\}, \{y^{(1)}, \dots, y^{(r)}\}) = \sum_t J^{(t)}$$

Example of RNN gradient evaluation

RNN gradients are evaluated using **back-propagation through time** (BPTT).

Considering a loss function where $\{y^{(1)}, \dots, y^{(\tau)}\}$ are targets:

$$J(\{x^{(1)}, \dots, x^{(\tau)}\}, \{y^{(1)}, \dots, y^{(\tau)}\}) = \sum_t J^{(t)}$$

As usual, starting from the last layer:

$$(\nabla_{o^{(t)}} J)_i = \frac{\partial J}{\partial o_i^{(t)}} = \frac{\partial J}{\partial J^{(t)}} \frac{\partial J^{(t)}}{\partial o_i^{(t)}} = \frac{\partial J^{(t)}}{\partial o_i^{(t)}}, \quad \frac{\partial J}{\partial J^{(t)}} = 1$$

11

Example of RNN gradient evaluation

RNN gradients are evaluated using **back-propagation through time** (BPTT).

Considering a loss function where $\{y^{(1)}, \dots, y^{(\tau)}\}$ are targets:

$$J(\{x^{(1)}, \dots, x^{(\tau)}\}, \{y^{(1)}, \dots, y^{(\tau)}\}) = \sum_t J^{(t)}$$

As usual, starting from the last layer:

$$(\nabla_{o^{(t)}} J)_i = \frac{\partial J}{\partial o_i^{(t)}} = \frac{\partial J}{\partial J^{(t)}} \frac{\partial J^{(t)}}{\partial o_i^{(t)}} = \frac{\partial J^{(t)}}{\partial o_i^{(t)}}, \quad \frac{\partial J}{\partial J^{(t)}} = 1$$

and using design 1 where $o^{(\tau)} = c + Vh^{(\tau)}$:

$$\nabla_{h^{(\tau)}} J = \frac{\partial J}{\partial h^{(\tau)}} = \frac{\partial o^{(\tau)}}{\partial h^{(\tau)}} \frac{\partial J^{(\tau)}}{\partial o^{(\tau)}} = V^T \nabla_{o^{(\tau)}} J$$

11

La differenza è che devo introdurre il concetto di tempo. La loss sarà la somma delle loss sui singoli tempi

Example of RNN gradient evaluation

Thus:

$$\nabla_{h^{(t)}} J = \left(\frac{\partial h^{(t+1)}}{\partial h^{(t)}} \right)^T (\nabla_{h^{(t+1)}} J) + \left(\frac{\partial o^{(t)}}{\partial h^{(t)}} \right)^T (\nabla_{o^{(t)}} J)$$

Using design 1, where $o^{(t)} = c + Vh^{(t)}$ and $h^{(t)} = \tanh(a^{(t)})$:

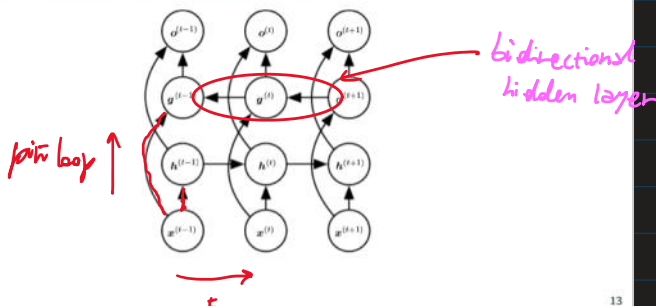
$$\nabla_{h^{(t)}} J = W^T \text{diag} \left(1 - (h^{(t+1)})^2 \right) (\nabla_{h^{(t+1)}} J) + V^T (\nabla_{o^{(t)}} J)$$

12

Posso quindi usare tecniche Hessiane/gradient descent

Bidirectional RNNs

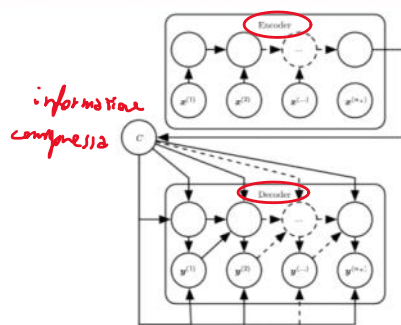
Information is propagated forward and backward in time.



13

Ci sono architetture un po' più complicate

Encoder-decoder sequence-to-sequence architectures

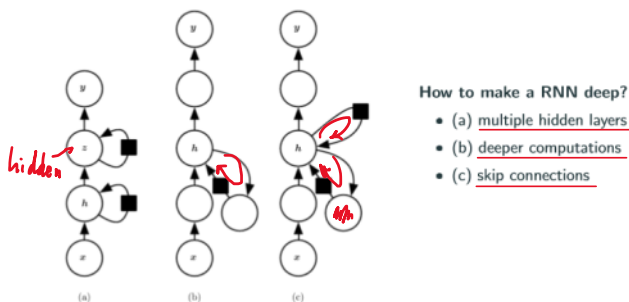


Compressione dell'informazione con correlazione

↪ elementi base per
informazione che viene propagata

Deep RNNs

Deep Recurrent Networks



Nei modellini prima ho pochi parametri

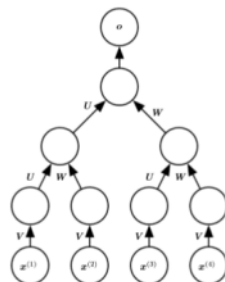
Per aumentarli posso introdurre più layer recurrent,
Posso poi avere diverse architetture (i.e. a,b,c)

recurrent

Recursive Networks

RvNN is another generalization of deep recurrent networks with:

- Structured predictions.
- Reduced depth.
- Potentially simpler to train.
- Successful applications in NLP.
(e.g. word embedding)



Più facile da allenare (come per esempio frasi) o anche calcoli più complicati tipo tensor networks (vedere il flusso di informazioni).

↪ buone

DRNNs issues

Deep RNN is designed to propagate "long-term" dependencies, $h^{(t)}$.

However, recurrent evaluations over many hidden layers deteriorate the gradient.

RNN gradients tend to vanish or diverge, for example:

$$\begin{aligned}\nabla_{\mathbf{W}} J &= \sum_t \sum_i \left(\frac{\partial J}{\partial h_i^{(t)}} \right) \nabla_{\mathbf{W}^{(t)}} h_i^{(t)} \\ &= \sum_t \text{diag} \left(1 - (h^{(t)})^2 \right) (\nabla_{h^{(t)}} J) h^{(t-1)T} \\ &\propto \prod_i \frac{\partial \sigma_h}{\partial h_i} \rightarrow 0 \quad \left(\text{with non-linear activation functions} \right)\end{aligned}$$

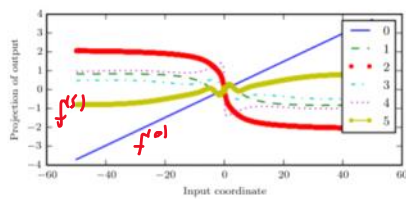
17

Se metto troppi step intermedi, il gradiente è proporzionale a a^* e se l'activation function è non lineare questa derivata diventa piatta. Quindi una rnn con molti parametri diventa difficile da allenare.

RNN con molti parametri tende a vanishing gradient, o se peggio diverge

DRNNs issues

Example: repeated nonlinear function composition.



RNN layers that get a small gradient update stop learning (usually earlier layers).

- RNNs **forget** information from long sequences.
- RNNs have a **short-term memory** issue.

18

Un'activation lineare? Diventa poi più piatta

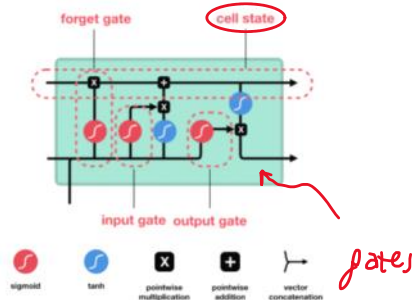
Per ovviare a questo, facciamo che ci dimentichiamo di informazione, e mitigiamo il problema di short term memory.

Problemi che affronto per poter mettere parametri nelle rnn

Fixing short-term memory

Long short-term memory

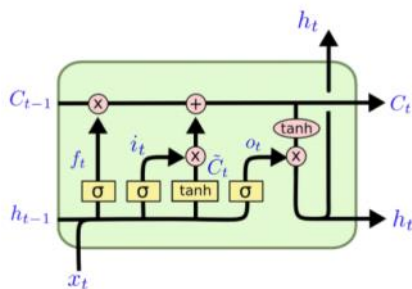
Long short-term memory (LSTM) networks introduce the concept of **gates** that regulate the flow of information.



19

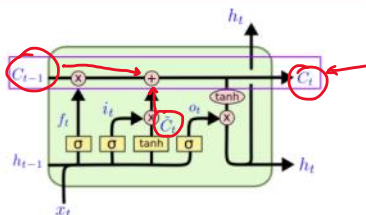
LSTM long short term memory, buoni risultati, una cella è un composto di operazioni dette gates, che dice quanta informazione voglio far passare

LSTM



20

LSTM cell state



The cell state stores the relevant information over time:

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t$$

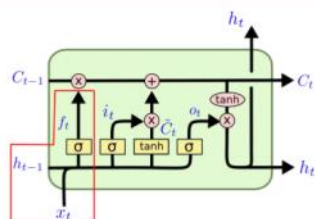
The **forget** and the **input** gates decide what we forget and remember.

21

Linea di propagazione dell'informazione nel tempo, Combinazione lineare al tempo t-1 e t

Parametri da allenare
 f_t forget gate, quanto dimenticare
 i_t quanto voglio ricordare
 $f_t \gg$ informazione passata
 $i_t \gg$ informazione attuale

LSTM forget gate



The **forget gate** decides what information we are going to forget and keep from cell state.

$$f_t = \sigma_g(W_f x_t + U_f h_{t-1} + b_f)$$

Takes the input x_t , hidden state h_{t-1} and $\sigma_g \in [0, 1]$ (sigmoid).

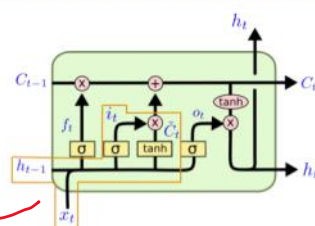
22

Dice quanta informazione voglio tenere

f_t a sua volta è funzione di altro

Abbiamo una sorta di piccola mlp

LSTM input state



The **input gate** chooses the new information to store in the cell state.

$$i_t = \sigma_g(W_i x_t + U_i h_{t-1} + b_i)$$

$$\tilde{C}_t = \sigma_c(W_c x_t + U_c h_{t-1} + b_c)$$

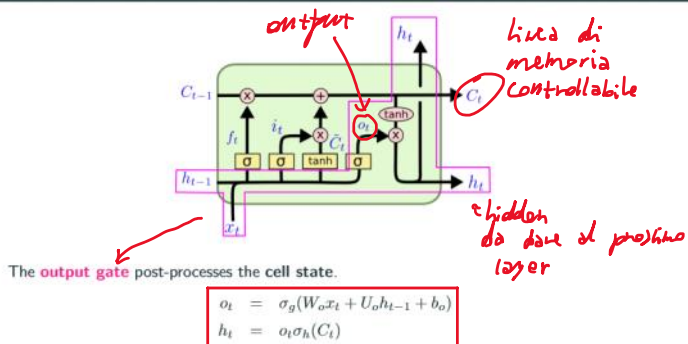
The sigmoid (σ_g) decides which values to update, the **tanh** (σ_h) builds the new candidate $\tilde{C}_t$.

23

Secondo pezzo, diciamo quanta nuova informazione vogliamo tenere e salvarla allo stato della cella. Il risultato viene iniettato nel cell state.

Normalmente si usa il sigmoide, mentre la Ctilda è la tanh. Queste si lasciano fissi, fatti dopo diversi studi

LSTM output state

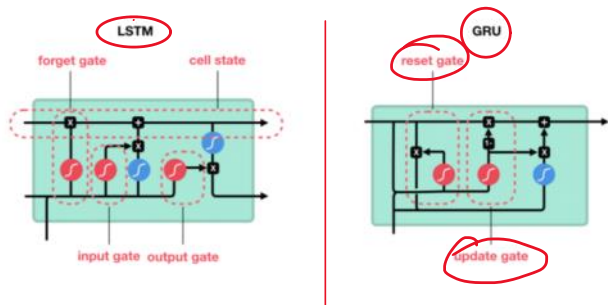


La cosa nuova, è che porto una linea C_t con caratteristica di modulazione di aggiornamento dell'informazione

Tengo in C tutte le correlazioni dei dati

Gated recurrent unit

: similar to LSTMs with a reset and update gates thus a smaller number of parameters.



Si è dimostrato che GRU è leggermente meglio a LSTM in alcuni casi

Unica architettura in cui l'informazione viene passata da un layer all'altro

Deep Learning - lecture 6

Convolutional Neural Networks

Prof. Stefano Carrazza

University of Milan and INFN Milan

Introduction to images

Images

Definition:

An **image** is an artifact that depicts visual perception.

1

Images

Definition:

An **image** is an artifact that depicts visual perception.

Some characteristics of images:

- two-dimensional (x, y)
- three-dimensional (x, y, z)

1

Images

Definition:

An **image** is an artifact that depicts visual perception.

Some characteristics of images:

- two-dimensional (x, y)
- three-dimensional (x, y, z) ← i.e. fisica medica

Devices used to capture images:

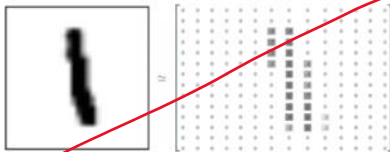
- *Natural objects and phenomena*: human eye, water
- *Optical devices*: cameras, mirrors, lenses, telescopes, microscopes

1

Digital images treatment

Nowadays images are converted to **digital images** representation.

Example: consider a simple image from the MNIST dataset:

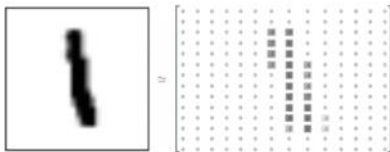


2

Digital images treatment

Nowadays images are converted to **digital images** representation.

Example: consider a simple image from the MNIST dataset:



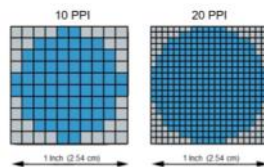
Digital images provide the possibility to:

- **Store and transfer** data images across computers.
- Apply and implement **image processing algorithms**.

2

Digital image representation

A digital image is an image composed of pixels, each with finite, discrete quantities of numeric representation for its intensity or gray level.



We can distinguish at least two properties:

- Image **resolution**: measure of pixels in the display (width x height).
- Image **density**: pixels per inch (PPI) or dot's per inch (DPI).

3

Immagine = artefatto della visione

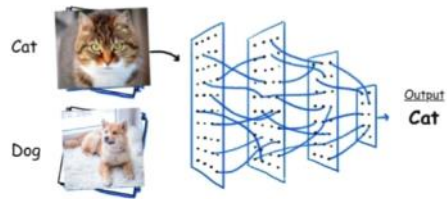
A noi interessa la versione digitale delle immagini

Posso definire un'immagine con lunghezza altezza(fisiche) e una risoluzione in pixel

Computer Vision Problems

Computer Vision Problems

Image Classification

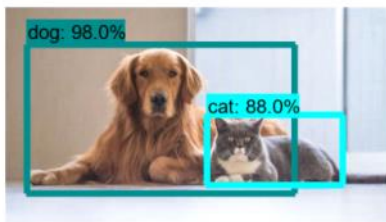


Vogliamo discriminare l'immagine

4

Computer Vision Problems

Object Detection



5

Computer Vision Problems

Neural Style Transfer



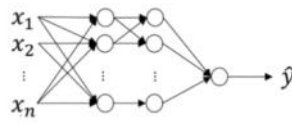
6

Reti per creare filtri

Deep Learning on large images



1000x1000x3 = 3M inputs



(1000, 3M) → 3 billion

7

Ma immagini grandi hanno molti parametri

Troppi parametri => perdita gradiente, e correlazione pixel vicini

Digital image transformations

Digital images treatment

Image processing techniques:

- affine and filtering transforms: *rotation, translations, denoising, ...*
- pattern recognition: *tasks related to human interpretation*

8

Dobbiamo ricordare un po' di cose. Se passo un'immagine posso o cliccare trasformazioni(i.e. filtri, rotazioni, traslazioni, rumore) o posso risolvere problemi di classificazione.

La prima famiglia di task le posso risolvere con algebra lineare, per le seconde ho task umane e ho bisogno di AI

Filtering transformations

Filtering transformations can be performed by:

- **Convolutions** with kernels, ω , in the spatial domain:

$$o(x, y) = \omega * f(x, y) = \sum_{dx=-a}^a \sum_{dy=-b}^b \omega(dx, dy) f(x + dx, y + dy)$$

where $o(x, y)$ is the filtered image and $f(x, y)$ the original image.

9

Filtering transformations

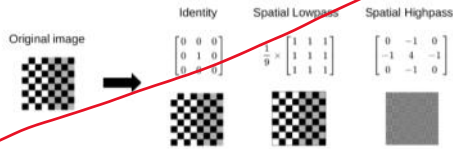
Filtering transformations can be performed by:

- **Convolutions** with kernels, ω , in the spatial domain:

$$o(x, y) = \omega * f(x, y) = \sum_{dx=-a}^a \sum_{dy=-b}^b \omega(dx, dy) f(x+dx, y+dy)$$

where $o(x, y)$ is the filtered image and $f(x, y)$ the original image.

Examples:



9

Filtering transformations

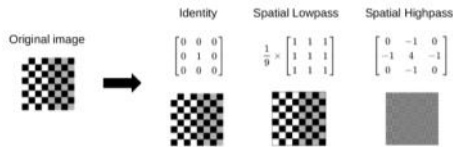
Filtering transformations can be performed by:

- **Convolutions** with kernels, ω , in the spatial domain:

$$o(x, y) = \omega * f(x, y) = \sum_{dx=-a}^a \sum_{dy=-b}^b \omega(dx, dy) f(x+dx, y+dy)$$

where $o(x, y)$ is the filtered image and $f(x, y)$ the original image.

Examples:



- Applying **masks** to frequency regions.

9

La convoluzione è un'operazione nota. A seconda della scelta del kernel ottengo un risultato diverso.

Applicando set di kernels, moltiplicati all'immagine ottengo i risultati sotto

- kernel
- siamo noti di

Affine transformations

Affine transformations are performed by applying the **affine matrix** to the pixel coordinates of a given image.

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix}_o = \begin{bmatrix} a & b & 0 \\ c & d & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}_f$$

where $o(x, y)$ is the filtered image and $f(x, y)$ the original image.

10

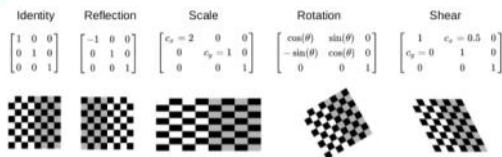
Affine transformations

Affine transformations are performed by applying the **affine matrix** to the pixel coordinates of a given image.

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix}_o = \begin{bmatrix} a & b & 0 \\ c & d & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}_f$$

where $o(x, y)$ is the filtered image and $f(x, y)$ the original image.

Examples:



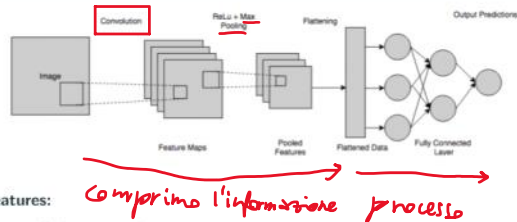
10

Le trasformazioni affini sono invece queste

Convolutional Networks

Nel caso di pattern recognition, la risposta migliore sono le CNN

Motivation



Major features:

- uses convolutions operations.
- contains at least one convolutional layer.
- mitigates large weight models
- several successful applications in 1-D and 2-D grid data

11

Convolution Operation

A convolution operation is defined as:

$$s(t) = (x * w)(t) = \int x(a)w(t-a)da$$

where $w(a)$ is a weighting function / probability density function.

$$\underbrace{s(t)}_{\text{feature map}} = \int \underbrace{x(a)}_{\text{input}} \underbrace{w(t-a)}_{\text{kernel}} da$$

Examples:

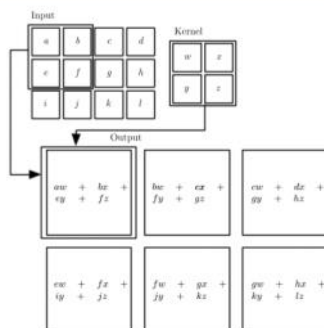
Input:		Kernel:		Output:																	
<table border="1"> <tr><td>0</td><td>1</td><td>2</td></tr> <tr><td>3</td><td>4</td><td>5</td></tr> <tr><td>6</td><td>7</td><td>8</td></tr> </table>	0	1	2	3	4	5	6	7	8	*	<table border="1"> <tr><td>0</td><td>1</td></tr> <tr><td>2</td><td>3</td></tr> </table>	0	1	2	3	=	<table border="1"> <tr><td>19</td><td>25</td></tr> <tr><td>37</td><td>43</td></tr> </table>	19	25	37	43
0	1	2																			
3	4	5																			
6	7	8																			
0	1																				
2	3																				
19	25																				
37	43																				

La convoluzione è definita come

- Output si riduce
- Comprimi l'informazione sommando i diversi elementi

12

Convolution Operation



Tutti i coefficienti sono quindi presenti su questa matrice finale.

13

Padding

Downsides of convolutions:

- size of the filtered image is small.
- corner pixels are not used as much as other pixels.

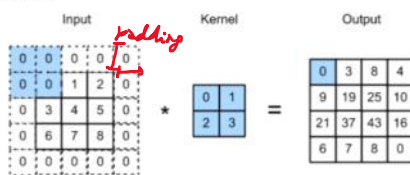
14

Padding

Downsides of convolutions:

- size of the filtered image is small.
- corner pixels are not used as much as other pixels.

Solution: add padding.



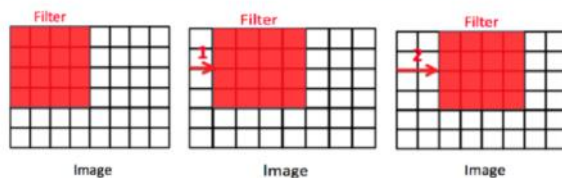
The output size is $(n + 2p - k + 1) \times (n + 2p - k + 1)$, with n the input size, k the kernel size, p the padding.

14

Il problema di questo tipo di approccio è che gli elementi al centro sono usati più volte che quelli esterni.

Per risolverlo uso del padding, in modo che l'informazione sui bordi sia comunque catturata

Strided convolution



Number of output features:

$$\left(\frac{n + 2p - k}{s} + 1 \right) \times \left(\frac{n + 2p - k}{s} + 1 \right)$$

where n is the input size, k the kernel size, p the padding and s the stride step.

15

Un'altra operazione che si fa è lo strides, che dice quanto distanziare i prodotti di convoluzione.

Per esempio posso vedere se riesce a catturare più informazioni

Se metto il kernel come parametro posso poi tunarlo e vedere il kernel migliore per darci l'informazione migliore

Advantages of a convolutional layer

Why a convolution layer?

Considering typical kernels smaller than the input:

- Sparse interactions (sparse weights/connectivity)
- Parameter sharing
- Equivariant representation

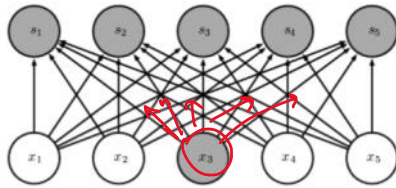
Furthermore, it is possible to combine convolutional layers with pooling and feed-forward layers.

16

- Abbiamo una riduzione della connettività
- I parametri sono condivisi tramite le convoluzioni
- Equivariant: siccome vogliamo comprimere l'immagine, stiamo dicendo che la convoluzione ci sta garantendo un'invarianza per rotazione e traslazione. Non importa dov'erano i pixel, il kernel (se allenato) le cattura

Sparse connectivity

Example of MLP with no sparse connectivity:



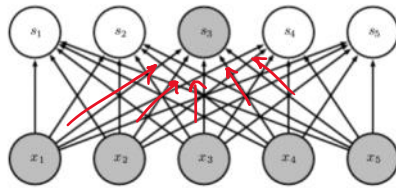
The input unit x_3 affects all outputs.

17

Sparse connectivity

Sparse connectivity

Example of MLP with no sparse connectivity:

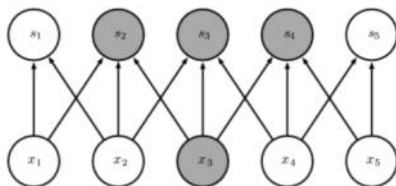


Each output unit s_3 is connected to all inputs.

18

Sparse connectivity

Example of sparse connectivity:



The input unit x_3 affects a subset of output units.

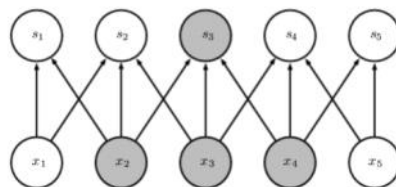
19

Qua non ho più tutti che parlano a tutti, ma posso avere un'architettura in cui le connessioni sono solo "locali", è più sparsa

Questo è vicino all'idea di convoluzione

Sparse connectivity

Example of sparse connectivity:

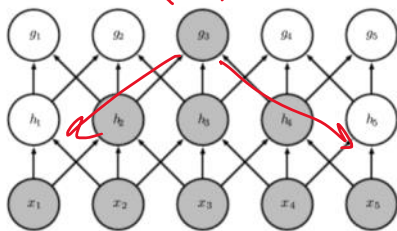


The input unit x_3 affects a subset of output units.

20

Sparse connectivity

Example of sparse connectivity:



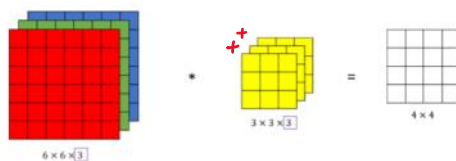
Even if connections are sparse, deeper layers are indirectly connected to most of the input.

21

C'è propagazione anche a distanze maggiori, ma in layer successivi. Limitiamo quindi le connessioni.

Convolutions on RGB images

We define a convolution kernel as a 3D tensor with a third dimension.

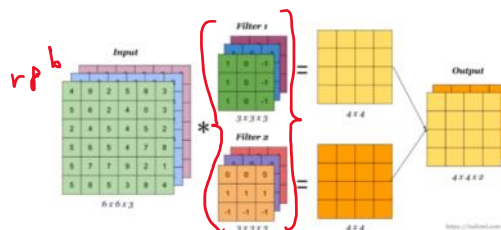


The convolution kernel is applied as a 3D volume.

22

Multiple kernels

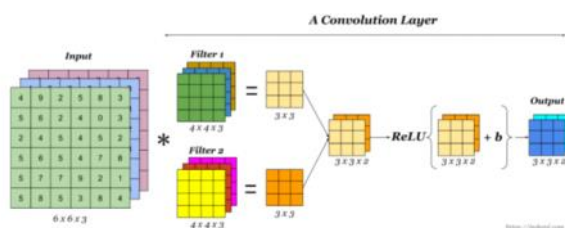
In order to extend the coverage of the filtering algorithm, we can apply multiple kernels to a given input and generate a tensor output.



23

Hyperparametri:
- Quanti filtri
- Quanto grandi
E poi anche padding e stride

Convolutional layer

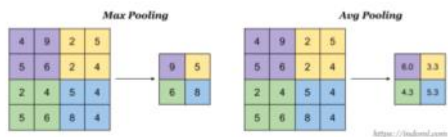


Finally, in order to increase the non-linearity in the output we apply a non-linear activation function in a convolution layer.

24

Pooling layer

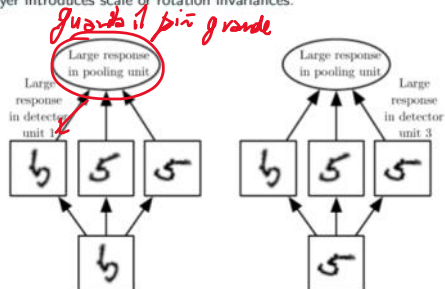
In a convolutional neural network, a convolutional layer is usually followed by a pooling layer. Pooling layer is usually added to speed up computation and to make some of the detected features more robust.



25

Pooling layer

The pooling layer introduces scale or rotation invariances.



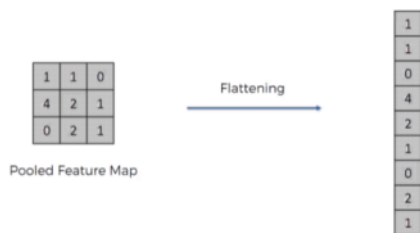
26

Un secondo stage è la compressione dell'informazione, che posso fare con il pooling

Il pooling layer garantisce che la CNN sia invariante per rotazioni (e la traslazione non proprio). Uno zoom in e out potresti avere qualche effetto.

Pooling dà un po' di stabilità

Flattening

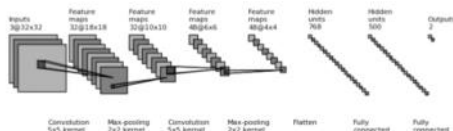


27

Lo step finale è il flattening, e parte poi un MLP

Fully connected layer

A fully connected layer is at the end of a convolutional neural network. The features map produced by the earlier layer is flattened to a vector.



28

Deep Learning - lecture 7

Transfer learning, object detection and segmentation

Prof. Stefano Carrazza

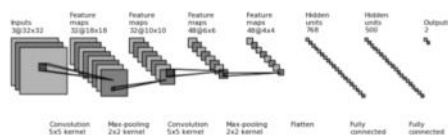
University of Milan and INFN Milan

What CNNs can do?

Image classification

Convolutional Neural Networks

- **Image classification:** predict class of object in an image.



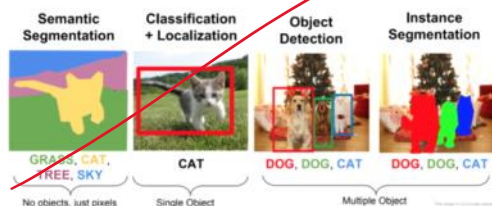
Convolutions **accelerate predictions** and **reduce the number of weights**.

1

Detection and segmentation

New potential applications based on CNNs:

- **Object Localization:** locate presence of object(s) in an image

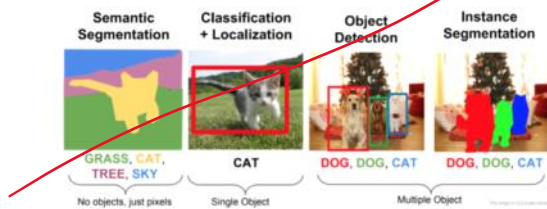


2

Detection and segmentation

New potential applications based on CNNs:

- **Object Localization:** locate presence of object(s) in an image
- **Object Detection:** locate and classify object(s) in an image.

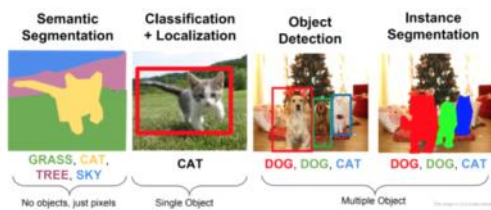


2

Detection and segmentation

New potential applications based on CNNs:

- **Object Localization:** locate presence of object(s) in an image
- **Object Detection:** locate and classify object(s) in an image.
- **Segmentation:** grouping pixels with comparable properties.



2

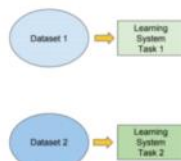
Come implemento questo dalle CNN?

Transfer Learning

Transfer learning idea

Traditional Deep Learning problem:

- Single task learning
- Large dataset
- Isolated problem
- **No knowledge is retained.**



C'è un concetto importante da fare prima. Il transfer learning è quando vogliamo evitare il problema di dover reimparare tutto ogni task diversa

Allenare può essere un bottleneck di cui tenere conto

3

The problem

Suppose 2 datasets, one big and another small, and a single task, e.g. image classification.

- How can we transfer the knowledge?
- Is it possible to reduce the training time?

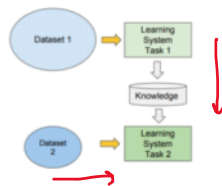
4

Per cui, dato un dataset grosso, se voglio propagare la conoscenza a diversi modelli, tra diversi dataset?

Transfer learning idea

Transfer learning:

- The learning process of a new task relies on the previous task.
- Requires less data.
- Training is faster and more accurate.



5

Sì, il ponte è detto di transfer learning

Transfer Learning

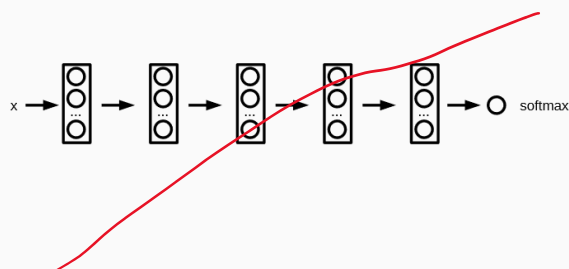
Transfer learning: a research problem that focuses on storing knowledge gained while solving one problem and applying it to a different but related problem.



6

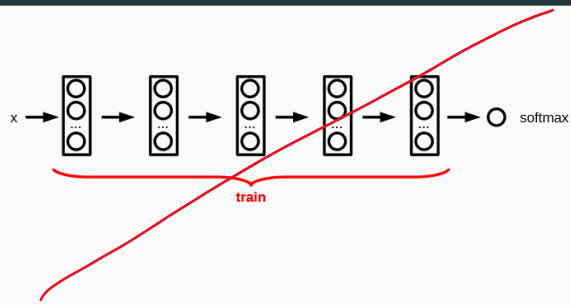
Ci sono più modi per farlo

Example



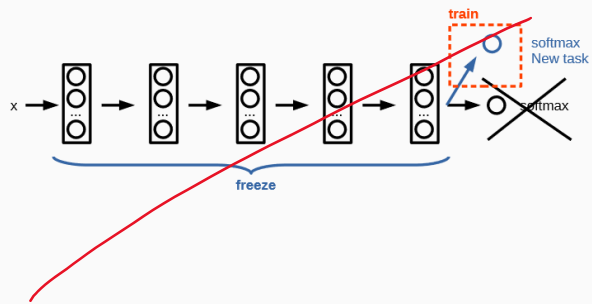
7

Example



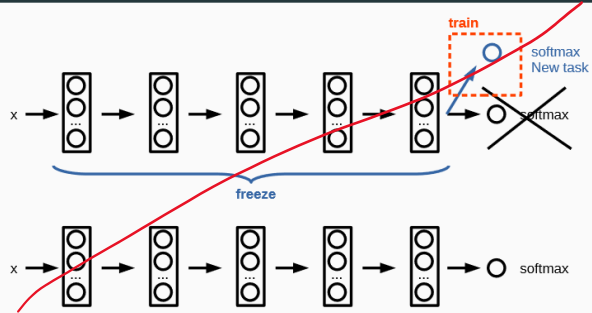
8

Example



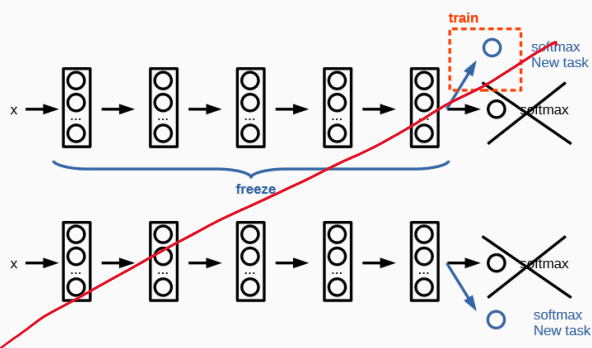
9

Example



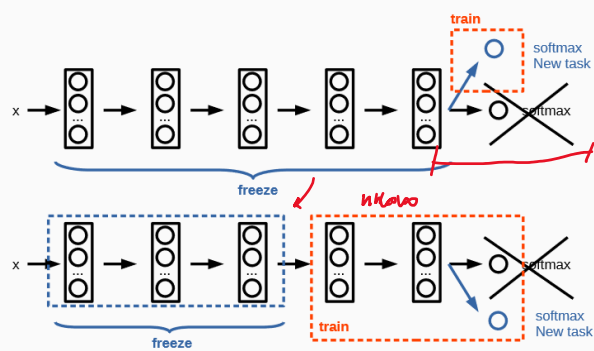
10

Example



11

Example



Primo allenamento che trova una buona scelta di parametri

Qua metto alla fine della rete un nuovo pezzo per fine tunare la rete per fare quello che voglio io nello specifico

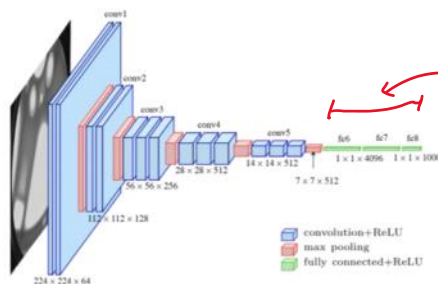
When transfer learning works?

- Task A and B have the same input.
- You have a lot more data for Task A than Task B.
- Low level features from A could be helpful for learning B.

Ovviamente, affinché funzioni, la task deve essere analoga, almeno della stessa tipologia, e l'input anche deve essere simile, e l'idea è quello di partire da un set di partenza più grande

Some examples of pre-trained models

- VGG-16
- VGG-19
- ResNet-50
- Inception V3
- ...



Quindi posso riciclare reti preallenate

In genere tocco giusto questi

Object localization and detection

Classification, localization vs detection

Classification



"car"

15

Classification, localization vs detection

Classification



"car"

Classification with localization



"car"

1 object

16

Classification, localization vs detection

cosa c'è?

Classification



"car"

+ dove?

Classification with localization



"car"

1 object

+ quanti?

Detection

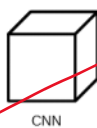


Multiple objects

17

Classification with localization

Classification



CNN

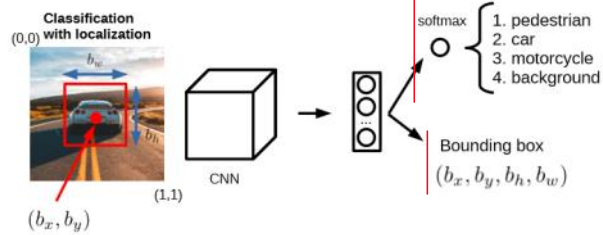


softmax

- 1. pedestrian
- 2. car
- 3. motorcycle
- 4. background

18

Classification with localization



Una parte mi dice cosa è, l'altra parte mi dice dove è

single loss function

19

Defining a target label

We can define a target output with 8 dimension (bounding box + 4 class labels):

$$y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

← c'è l'oggetto?

due

c'è

- p_c : 1 if object is detected, 0 otherwise
- b_x, b_y, b_h, b_w : bounding box coordinates.
- c_1, c_2, c_3 : class labels (pedestrian, car, motorbike).

With y , we can apply a loss function, for example if $y_1 = 1$ MSE for all components.

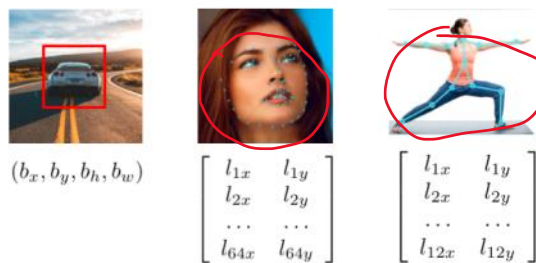
20

Defining a target label



21

Landmark detection



Versioni più evolute dei bounding box, dividendo l'oggetto in punti

Landmark detection

22

Object detection

Step 1) train CNN classifier on cropped images.



23

Capire se c'è

Object detection

Step 2) apply classifier on sliding windows.



24

Object detection

Step 2) apply classifier on sliding windows.



Major issue: high computational cost, not ideal for real-time detection.

24

Capire quante sono.

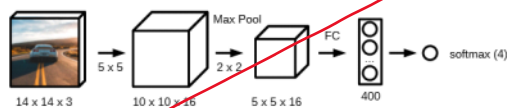
Per farlo posso per esempio avere un df croppato di immagini, dicendo se nella crop ho la macchina

Posso ho farla slidare, o usare reti come prima, ed adattare la grandezza in base a se catturo o meno la macchina

Ovviamente questo è inefficiente

Convolutional implementation sliding windows

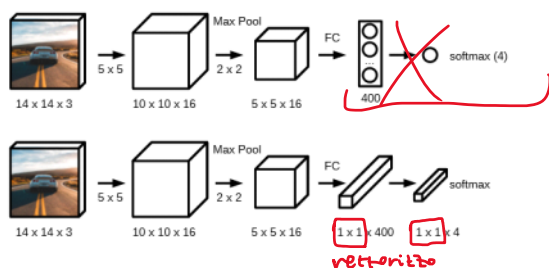
How to accelerate sliding windows? → Convert a FC layer into convolutional layer.



25

Convolutional implementation sliding windows

How to accelerate sliding windows? → Convert a FC layer into convolutional layer.

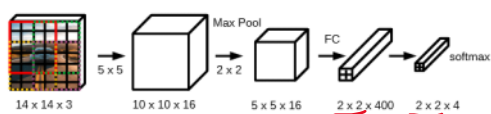


26

Posso poi riscrivere lo sliding, in un'ottica di CNN, in cui non faccio il reshape dell'immagine, ma parallelizzo con più channel

Convolutional implementation sliding windows

It is possible to define a convolution implementation of sliding windows.

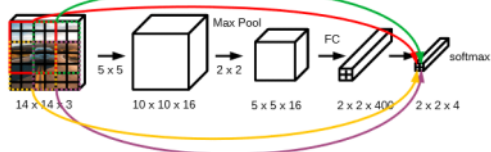


27

Con questo setup ho diverse predzioni che contemporaneamente vengono calcolati

Convolutional implementation sliding windows

It is possible to define a convolution implementation of sliding windows.

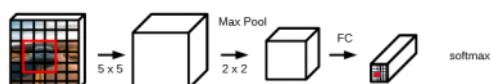


28

Quindi vettorizzare la predizione è un punto di forza

Convolutional implementation sliding windows

It is possible to define a convolution implementation of sliding windows.



29

Evaluating object localization



Intersection over Union (IoU)

Measures the overlap between two bounding boxes.

$$\text{IoU} = \frac{\text{Intersection area}}{\text{Union area}}$$

Correct object localization if $\text{IoU} \geq \text{threshold}$ (e.g. 0.5).

30

Quindi abbiamo tolto il problema del tempo.

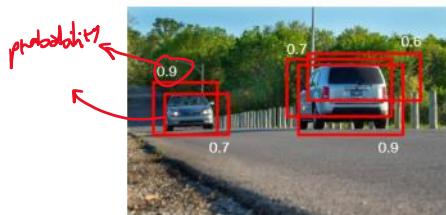
A questo punto bisogna capire quali tipi di bounding box guardo. Per esempio ho 2 bounding boxes che entrambe vedono la macchina. Di solito per capire se l'oggetto esiste, calcolo l'area di intersezione rispetto all'unione. Se è sopra un certo rate probabilmente c'è un oggetto, quindi c'è e stanno descrivendo lo stesso oggetto. Quindi per esempio posso considerare la soluzione l'intersezione

Non-max suppression



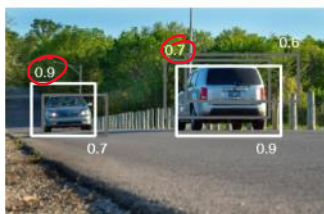
31

Non-max suppression



32

Non-max suppression



Take the largest p_c , compute the IoU for overlapping bounding boxes.

33

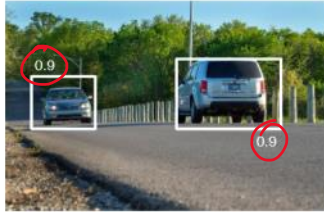
E quale probabilità uso?

Per esempio posso prendere il massimo

Non-max suppression



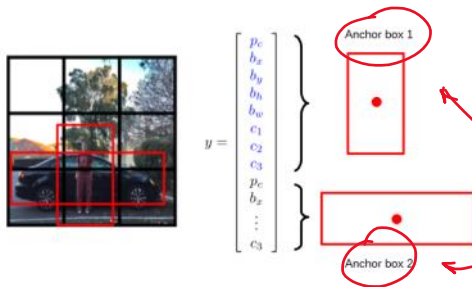
Non-max suppression



Non-max discards rectangles with large IoU.

34

Overlapping objects



35

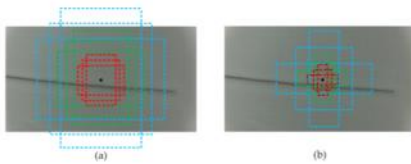
Se ho oggetti overlapping, posso solo che introdurre questo nel mio dataset, e del permetterlo. Attacco un pezzo nel vettore per capire se ho un oggetto davanti

2 bounding boxes

Anchor box algorithms

So far each object in training image is assigned to grid cell that contains that object's midpoint.

With two anchor boxes: each object in training image is assigned to grid cell that contains object's midpoint and anchor box for the grid cell with highest IoU.



36

Posso poi usare anchor box algorithms, ogni oggetto è assegnato ad una di queste celle, che contiene il centro dell'anchor box.

Queste tecniche mi dicono se sto prendendo delle buone anchor boxes, o sono troppo grandi o altro.

Accelerating object detection

Region proposals

Problem: avoid sliding windows as much as possible → increase performance.

How? → using a region proposal method, such as:

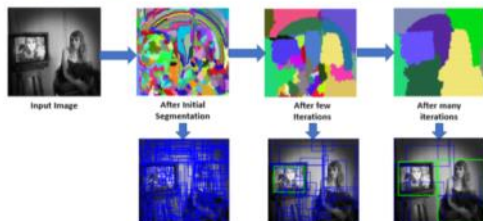
- Selective Search
- Randomized Prim
- Category independent object proposals
- Objectness
- Constrained parametric min-cuts for automatic object segmentation
- ...

37

Example: selective search

Selective Search:

1. Generate initial sub-segmentation of input image.
2. Use greedy algorithm to combine smaller similar regions into larger ones.
3. Use the segmented region proposal to generate candidate object locations.



38

Per esempio la **selective search**

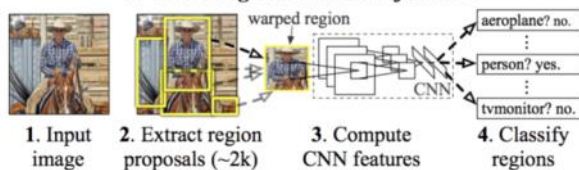
- Segmento intensificando il colore dei pixel e tonalità
- Cerco poi di agglomerare regioni

Dopo iterarlo arrivo ad un'immagine più squadrata a cui poi applicare il bounding box

Questo non è un modello AI based (ossia reti che ci dicono cosa fare, ma il vantaggio è che non devo allenarlo)

R-CNN: Regions with CNN features

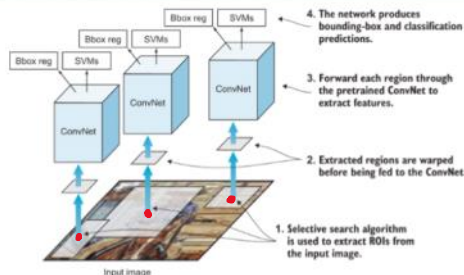
R-CNN: Regions with CNN features



Girshick et al., 2013, arXiv:1311.2524

39

R-CNN: Regions with CNN features

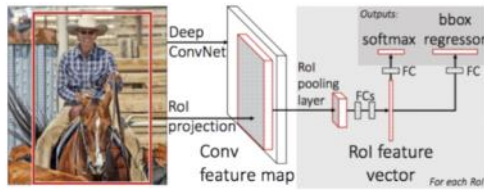


Issues: still **slow** (cannot be used for real time), relies on selective search.

40

Questo è comunque molto lento

Fast R-CNN

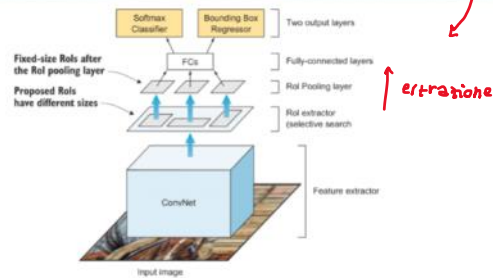


Girshick et al., 2015, arXiv:1504.08083

41

Fast RCNN, qua prendo l'immagine totale e la passo alla rete, che quindi gestisce tutto

Fast R-CNN



Use a convolution operation once per image → avoid feeding 2000 regions proposals.

42

Diagramma finale

estrazione

Fast R-CNN

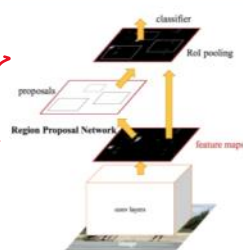


Fast R-CNN is faster in training and testing sessions over R-CNN.
Region proposals are the performance bottlenecks in Fast R-CNN.

43

Faster R-CNN

meno bene
(2 region proposal)



Replaces selective search with a region proposal network.

Shaoqing Ren et al., 2015, arXiv:1506.01497

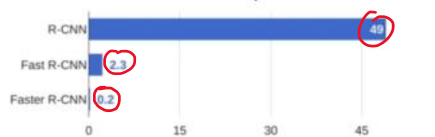
44

faster

State of the art

Faster R-CNN

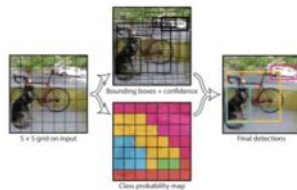
R-CNN Test-Time Speed



Faster R-CNN can be used for real-time object detection.

45

YOLO: you only look once



Redmon et al., 2015, arXiv:1506.02640

YOLO splits into a grid of bounding boxes. Each bounding box is associated to a class probability and offset values of the bounding box. The bounding boxes with class probability above a threshold are selected and used to locate the object.

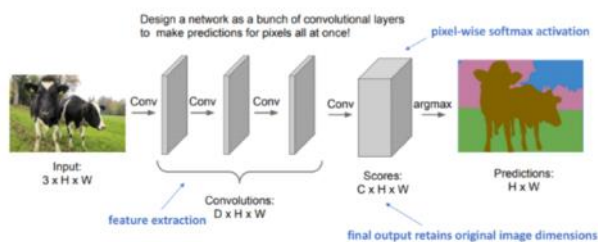
46

Oltre la faster RCNN, c'è YOLO, in cui faccio una griglia, che al posto del selective search, splitta e fa un class probability net per dire cosa si trova dove si trova.

Molto efficiente

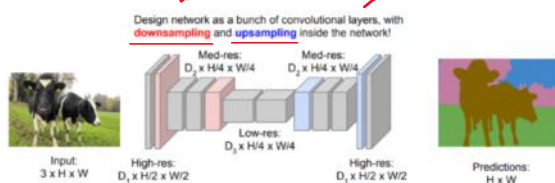
Instance segmentation

Semantic segmentation



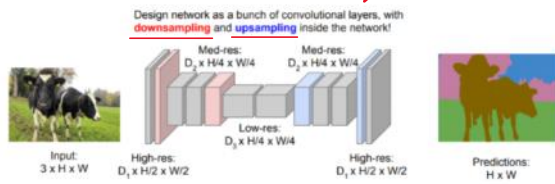
47

Semantic segmentation



Non ha ancora performance ottimali

Semantic segmentation

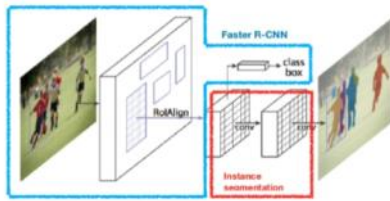


Solution: Make network deep and work at a lower spatial resolution for many of the layers.

48

Non ha ancora performance ottimali

Mask R-CNN



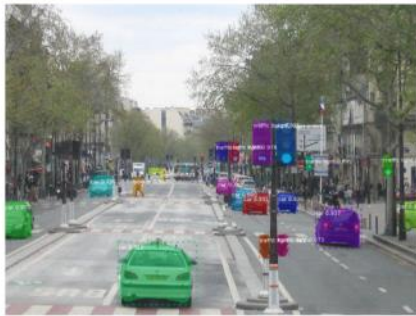
49

Simile alla Fast RCNN, ma infilando nel modello un layer in cui faccio convoluzioni pixel per pixel.

Combino con la FCRNN una CNN, ma facendoli lavorare solo sulle regioni buone, quindi taglio calcoli.

Uno dei modelli più usati per fare segmentazione completa

Mask R-CNN



50

Libreria che trovo online, in modo molto efficiente.
Utile per transfer learning

Github(librerie importanti usate):

- [Detectron2](#)
- [Mmdetection](#)

Deep Learning - lecture 8

Unsupervised learning, VAE and GAN

Prof. Stefano Carrazza

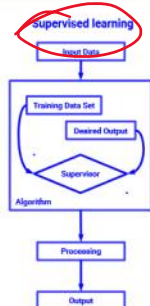
University of Milan and INFN Milan

Unsupervised learning

Supervised vs unsupervised learning

Supervised Learning:

- **Data and labels** are available.
- **Examples:** regression, classification, localization, object detection, segmentation, etc.



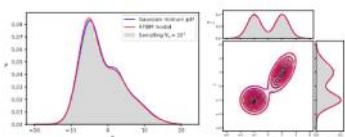
L'unsupervised learning è l'altro paradigma usato. E ci sono vari modelli sviluppati nell'ultimo decennio.

Tecnica solita, in cui di base so risolvere il mio problema

Supervised vs unsupervised learning

Unsupervised Learning:

- **Data and no labels!**
- **Examples:** density estimation, clustering, dimensionality reduction, etc. **i.e. PCA**



Unsupervised learning



Magari non ho neanche output ben precisi.

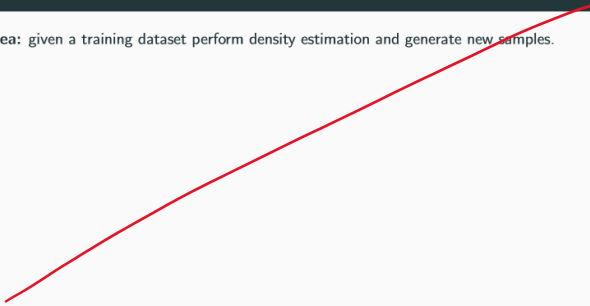
Per esempio la density estimation. Qual è la densità matematica che descrive questo programma? Un fit magari non va bene perché non è normalizzato, o è vincolato al numero di misure.

Nell'unsupervised, usando la presenza di dati imparo. estraggo proprietà che potrebbero di base non essere quelli che gli passo

Generative models

Generative models

Idea: given a training dataset perform density estimation and generate new samples.



3

Generative models

Idea: given a training dataset perform density estimation and generate new samples.

Solution: Learn $p_{\text{model}}(x)$ from $p_{\text{data}}(x)$

1. Given a training data with $p_{\text{data}}(x)$:



2. Estimate a model, $p_{\text{model}}(x)$, and generate samples:



Implement models for density estimation.

3

Why generative models?

- Realistic samples for computer vision applications: Generate Realistic Photographs, Generate Cartoon Characters, Image-to-Image Translation, Text-to-Image Translation, Semantic-Image-to-Photo Translation, Face Frontal View Generation, Generate New Human Poses, Photograph Editing, Face Aging, Photo Blending, Super Resolution, Photo Inpainting, Clothing Translation, Video Prediction, 3D Object Generation, etc...



- Data augmentation

Alec Radford et al. 2016, David Berthelot et al. 2017, Philip Isola et al. 2017

4

Voglio generare dati partendo da un campione input.

Per esempio voglio un modello matematico che riproduca la densità di probabilità di un dataset.

Le immagini sono densità di probabilità, perché i pixel sono più o meno illuminati in base alla funzione. Quindi se riesco a farlo, ho modelli che possono generare immagini simili a quelle di input.

Quindi posso tornare ai fit, imparando una densità che fitta i miei dati.

Il passaggio dal pdata che posso misurare, voglio passare al pmodel che ci dà la versione "fake" realistica dei dati.

Alcuni esempi di modelli generativi, sono:

- Generare immagini con disposizioni dei mobili
- Facce false
- Modello che da un disegno la colora

Autoencoders

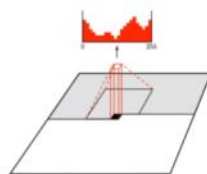
Explicit tractable density model

Lets consider the explicit tractable density model:

$$p(x) = \prod_{i=1}^n p(x_i | x_1, \dots, x_{i-1})$$

- $p(x)$ likelihood of e.g. image x .
- $p(x_i | x_1, \dots, x_{i-1})$ probability of i 'th pixel given all previous pixels.

Task: maximize likelihood of training data.



PixelCNN, van der Oord et al. 2016

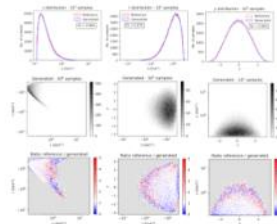
5

Explicit approximate density model

Lets consider the explicit intractable density model:

$$p_{\theta}(x) = \int p_{\theta}(z) p_{\theta}(x|z) dz$$

- θ model parameters
- z latent space
- Variational Autoencoders (VAEs) define intractable density function which cannot be optimized directly.

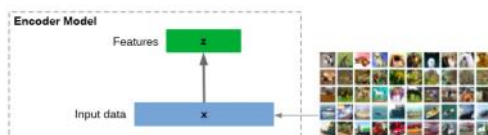


VAE, GAN, etc.

6

Autoencoders

Design an encoder model which learns how to represent unlabeled training data in a lower-dimensional space.



- z is the latent space, usually smaller than x .
- The encoder model can a linear and/or nonlinear model, deep FC, CNNs.

7

Facciamo uno step indietro per capire come fare queste operazioni. Un primo modello che parla di probabilità sono gli autoencoder.

Data un'immagine, composta da tanti pixel, per ogni pixel posso costruire una distribuzione di probabilità dell'intensità, che posso histogrammare. Posso definire una densità di pixel come $p(x)$ e dico che la likelihood finale è il prodotto di queste funzioni. Se sono scelte in modo analitico, la $p(x)$ diventa trattabile(implementabile). Posso poi calcolare la cumulativa, l'inversa, e poter poi campionare. Però ho dei vincoli.

Quindi, per avere $p(x)$, mi servono le condizionate $p(x|...)$. Se ogni pixel ha un comportamento diverso, il vincolo è troppo forte e rovina la predizione.

Per migliorare posso andare al continuo. Considero una variabile di integrazione latente z , e ho $p(z)$ e $p(x|z)$.

Questo è comunque intrattabile perché non ho una forma chiusa per l'integrale. È molto faticoso trovare un modello che lo fitti bene.

Dobbiamo quindi cercare modelli parametrici, che numericamente siano in grado di generare dati.

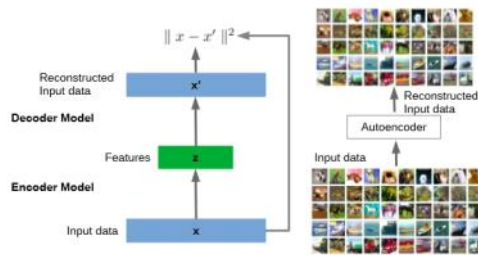
Un autoencoder ha:

- Input data x (filtrati magari)
- Proietta l'informazione su un sottospazio latente z più piccolo

Questo passaggio è non lineare, e quindi in grado di rimuovere la ridondanza, ma sia descrittivo, in modo da saper riprodurre le immagini.

L'encoder codifica quindi l'informazione in uno spazio minore.

Autoencoders



8

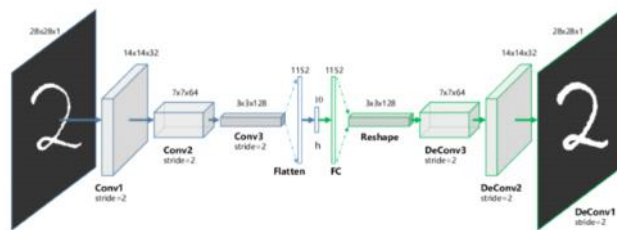
Se abbiamo implementato bene l'encoder, posso generare un'inversa, detto decoder, che dal feature space ridotto, mi riporta allo spazio originale.

La complessità è trovare un modello in grado di comprimere ed estrarre le informazioni in modo efficiente.

Volendo potrei fare poi algoritmi all'interno dello spazio ridotto, e che vada a generare fluttuazioni sui dati di input, e vado quindi a generare dati diversi.

Se disturbo z in modo completamente random nessuno mi garantisce non ottenga spazzatura.

Autoencoders

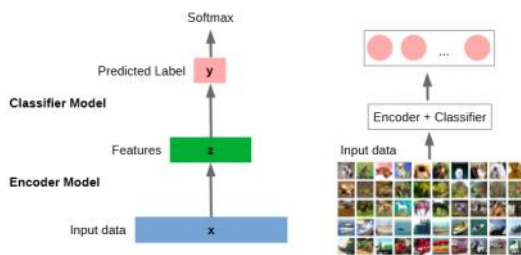


9

Partendo dall'idea dell'encoder, per esempio anche in supervised learning, ha un contenuto informativo alto, e quindi ho una tecnica per allenare l'encoder buona?

Autoencoders application example

Encoder can be used to initialize a supervised model, e.g. classifier:



10

Variational Autoencoder

Motivation

- Autoencoders learn how to reconstruct data.
- The latent space captures variations in the training data and removes redundancy.
- Can we generate new images/samples from an autoencoder?

⇓

Variational autoencoders (VAE)

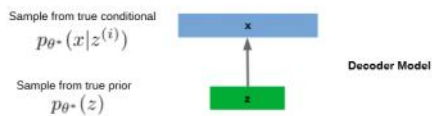
In questo caso andiamo a migliorare il discorso, dicendo che gli autoencoders, non solo ricostruiscono i dati, ma:

- Lo spazio latente deve catturare tutta l'informazione e rimuovere le ridondanze
- Saper generare nuovi dati

11

Variational autoencoders

Assuming training data $\{x^{(i)}\}$ is generated from underlying unobserved (latent) representation.

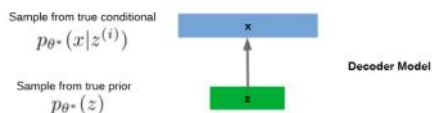


Target goal: estimate the true parameters θ^* of this generative model.

Kingma and Welling, Auto-Encoding Variational Bayes, ICLR 2014

12

Variational autoencoders



In practice the decoder network is composed by:

- A simple (closed-form) prior $p(z)$, e.g. Normal distribution.
- A Neural Network for $p(x|y)$.

13

How to train a VAE?

Maximize likelihood of training data:

$$p_{\theta}(x) = \int p_{\theta}(z)p_{\theta}(x|z)dz$$

However, the integral is **intractable**, i.e. to compute $p(x|z)$ for every z . (no closed form)

14

How to train a VAE?

Maximize likelihood of training data:

$$p_{\theta}(x) = \int p_{\theta}(z)p_{\theta}(x|z)dz$$

However, the integral is **intractable**, i.e. to compute $p(x|z)$ for every z . (no closed form)

Posterior density is also **intractable**: $p_{\theta}(z|x) = p_{\theta}(x|z)p_{\theta}(z)/p_{\theta}(x)$

14

How to train a VAE?

Maximize likelihood of training data:

$$p_{\theta}(x) = \int p_{\theta}(z)p_{\theta}(x|z)dz$$

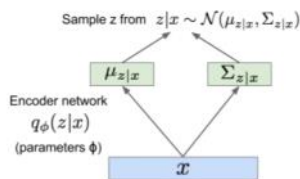
However, the integral is **intractable**, i.e. to compute $p(x|z)$ for every z . (no closed form)

Posterior density is also **intractable**: $p_{\theta}(z|x) = p_{\theta}(x|z)p_{\theta}(z)/p_{\theta}(x)$

Potential solution: include an additional encoder network that approximates $p_{\theta}(z|x)$.

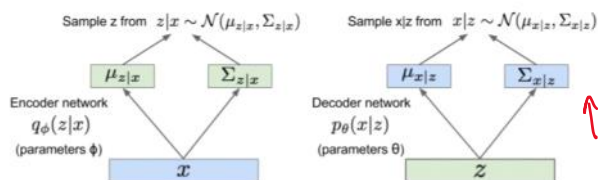
14

Probabilistic generation of data



15

Probabilistic generation of data



↑ completamento
↳ ricostruzione
l'informazione
originale

16

Quindi, per il prior di z si usa una gaussiana, e il passaggio dalla distribuzione normale allo spazio finale, sarà dato da una rete neurale.

Questi sono i 2 requirements per creare dati:

- Parto da una distribuzione snessata
- La uso per tornare nella cosa? Originale

L'integrale sarà sempre intrattabile (non ha una formula chiusa). Anche la posteriore density è intrattabile perché ha $p(z)$ dentro. Ma posso usare una $p(z|x)$? come un'altra rete

Quello che ci mancava era quindi, per costruire la $p(z)$ prendiamo i dati di input. Costruiamo una rete neurale, con il compito che per ogni x , prende x e dà come output parametri utili per la $p(z)$. Se è normale, posso chiedere alla rete di generare media e std dev per ogni input.

Non è quindi una rete banale che prende e comprime, ma cattura le correlazioni, per avere una costruzione del decoder coerente?

Maximizing likelihood

$$\begin{aligned}
 \log p_{\theta}(x^{(i)}) &= \mathbb{E}_{z \sim q_{\phi}(z|x^{(i)})} [\log p_{\theta}(x^{(i)})] \\
 &= \mathbb{E}_z \left[\log \frac{p_{\theta}(x^{(i)}|z)p_{\theta}(z)}{p_{\theta}(z|x^{(i)})} \right] \\
 &= \mathbb{E}_z [\log p_{\theta}(x^{(i)}|z)] - \mathbb{E}_z \left[\log \frac{q_{\phi}(z|x^{(i)})}{p_{\theta}(z)} \right] + \mathbb{E}_z \left[\log \frac{q_{\phi}(z|x^{(i)})}{p_{\theta}(z|x^{(i)})} \right] \\
 &= \underbrace{\mathbb{E}_z [\log p_{\theta}(x^{(i)}|z)]}_{\text{log likelihood}} - \underbrace{D_{KL}(q_{\phi}(z|x^{(i)}), p_{\theta}(z))}_{\text{dist. distributioni}} + \underbrace{D_{KL}(q_{\phi}(z|x^{(i)}), p_{\theta}(z|x^{(i)}))}_{\text{distance encoder decoder}}
 \end{aligned}$$

17

Maximizing likelihood

$$\begin{aligned}
 \log p_{\theta}(x^{(i)}) &= \mathbb{E}_{z \sim q_{\phi}(z|x^{(i)})} [\log p_{\theta}(x^{(i)})] \\
 &= \mathbb{E}_z \left[\log \frac{p_{\theta}(x^{(i)}|z)p_{\theta}(z)}{p_{\theta}(z|x^{(i)})} \right] \\
 &= \mathbb{E}_z [\log p_{\theta}(x^{(i)}|z)] - \mathbb{E}_z \left[\log \frac{q_{\phi}(z|x^{(i)})}{p_{\theta}(z)} \right] + \mathbb{E}_z \left[\log \frac{q_{\phi}(z|x^{(i)})}{p_{\theta}(z|x^{(i)})} \right] \\
 &= \underbrace{\mathbb{E}_z [\log p_{\theta}(x^{(i)}|z)]}_{\text{log likelihood}} - \underbrace{D_{KL}(q_{\phi}(z|x^{(i)}), p_{\theta}(z))}_{\text{dist. distributioni}} + \underbrace{D_{KL}(q_{\phi}(z|x^{(i)}), p_{\theta}(z|x^{(i)}))}_{\text{distance encoder decoder}}
 \end{aligned}$$

• $J(x^{(i)}, \theta, \phi)$ is a tractable lower bound
 • $D_{KL}(q_{\phi}(z|x^{(i)}), p_{\theta}(z)) > 0$

17

Maximizing likelihood

We have a variational lower bound (ELBO):

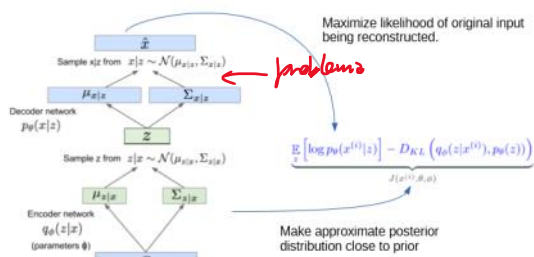
$$\log p_{\theta}(x^{(i)}) \geq J(x^{(i)}, \theta, \phi)$$

thus the training should maximize:

$$\theta^*, \phi^* = \arg \max_{\theta, \phi} \sum_{i=1}^N J(x^{(i)}, \theta, \phi)$$

18

Maximizing likelihood



19

Se l'obiettivo è quello di massimizzare la log likelihood, manipolando l'espressione, ho una loss function finale

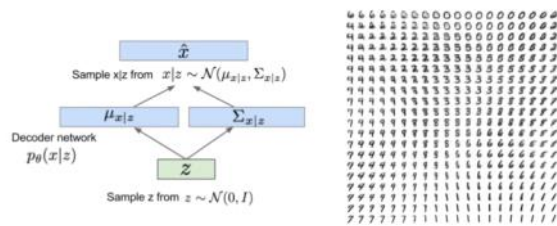
Se quindi alleno, devo massimizzare la somma delle loss function di tutti i pixel.

Questo si traduce in un problema classico di deep learning in cui massimizzo la log likelihood

Alleno quindi tutto insieme. Quasi tutti i casi che uso i VAE uso le gaussiane. Questi parametri sono poi usati per campionare. La predizione finale $\hat{x}$ entra in J , mentre la prima parte entra nella D_{KL} .

Qual è il problema di questo modello? Una linea potrebbe giustificare perché non sono così usate le VAE. Prima della $\hat{x}$ c'è ancora una gaussiana. Quindi i risultati non saranno perfetti

Sampling VAE



20

In alcuni casi l'immagine è un po' sfocata

Generating data



32x32 CIFAR-10



Labeled Faces in the Wild

21

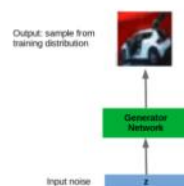
Per cose più complesse questo diventa più importante.

Questo è il primo algoritmo variational in grado di produrre immagini fake.

Generative Adversarial Networks

Generative Adversarial Networks (GANs)

Problem: sample a complex, high-dimensional training distribution.



Idea: learn how to transform random noise into sampling from a training distribution.

I. Goodfellow et al. "Generative Adversarial Nets", NIPS 2014

22

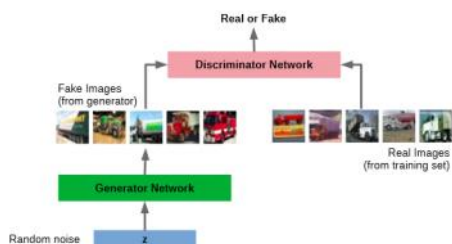
Si è andati su altre possibilità per correggere questo comportamento. Le GAN. In questo caso il problema diventa di battere le VAE. Il vincolo modellistico era la gaussiana. Vogliamo quindi sia migliorarlo, che scalare. È quindi stato presentato per la prima volta nel 2014.

Lo spazio latente z diventa ora un input noise. Questo entra nella rete che fa da generatore e impara a ricostruire l'immagine. Questo toglie il problema della VAE della gaussiana, e la rete impara da sé i vincoli e tutto.

GANs: Two-player game

We define 2 networks:

- **Generator network:** try to fool the discriminator by generating real-looking images.
- **Discriminator network:** try to distinguish between real and fake images.



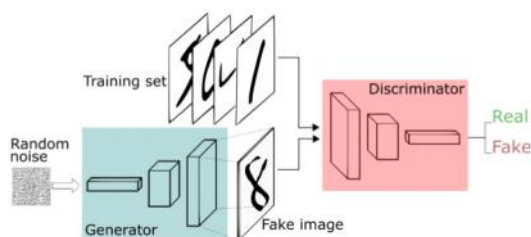
23

Per allenarlo, si introducono reti. Sicuramente la generativa di prima. Poi ho una rete di discriminatore che uso come classificatore. La rete deve imparare la prior, il discriminatore invece cerca di indovinare le cose fake, e spinge quindi il generatore a fare output migliori. Il generatore da solo non è in grado a capire da solo se i suoi output sono vicino a quelli originali.

Inoltre non vogliamo fare una cosa supervised, che impara le densità delle immagini di input. Voglio un dataset diverso da quello iniziale, ma che preservi le caratteristiche di quello originale.

Qua non devo calcolare metriche strane tipo D_KL tra immagini. Qui usando il discriminatore dice giusto se l'output è fake o vero.

GANs: Two-player game



24

Training a GAN

Train jointly in minmax game:

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{\text{data}}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log (1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

Discriminator, θ_d :

Maximizes objective such that:

- $D_{\theta_d}(x) \rightarrow 1$ (real)
- $D_{\theta_d}(G_{\theta_g}(z)) \rightarrow 0$ (fake)

compito del discriminatore

Generator, θ_g :

Minimizes objective such that:

- $D_{\theta_d}(G_{\theta_g}(z)) \rightarrow 1$
- Discriminator thinks $G_{\theta_g}(z)$ is real.

25

Per il training, per la loss function faccio uno splitting, una parte dedicata al discriminatore, e una log likelihood, dove G è la densità che fa riferimento al generatore.

Training a GAN

Minmax objective function:

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{\text{data}}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log (1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

Training procedure, alternate the following steps:

1. Perform **gradient ascent** on discriminator:

$$\max_{\theta_d} \left[\mathbb{E}_{x \sim p_{\text{data}}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log (1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

2. Perform **gradient descent** on generator:

$$\min_{\theta_g} \left[\mathbb{E}_{z \sim p(z)} \log (1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

Usually replaced by a gradient ascent: $\max_{\theta_g} \mathbb{E}_{z \sim p(z)} \log (D_{\theta_d}(G_{\theta_g}(z)))$

26

Quindi quando alleniamo dividiamo in due parti.

Un gradient ascent che massimizza il primo termine, e dopo qualche epoca passo al secondo step e itero.

Training a GAN

for number of training iterations do
for k steps do

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_z(z)$.
- Sample minibatch of m examples $\{x^{(1)}, \dots, x^{(m)}\}$ from data generating distribution $p_{\text{data}}(x)$.
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m [\log D_{\theta_d}(x^{(i)}) + \log(1 - D_{\theta_d}(G_{\theta_g}(z^{(i)})))]$$

end for

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_z(z)$.
- Update the generator by ascending its stochastic gradient (improved objective):

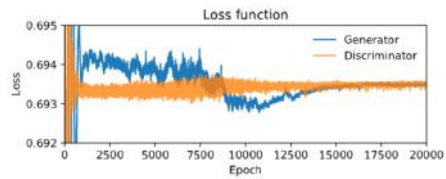
$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log(D_{\theta_d}(G_{\theta_g}(z^{(i)})))$$

end for

27

Questo è l'algoritmo

Training a GAN



28

Normalmente il generatore è quello che fa più fatica, quindi grafici di questo tipo sono normali.

Qua fluttua tanto quindi non si vede, la media del generatore? si alza un pochettino quando inizia a generare molto bene.

Examples of GAN samples



29

Non ho più l'effetto blurred. Dentro il box giallo sono le originali, e le altre sono quelle generate.

Examples of GAN samples



30

Examples of GAN samples



31

Arredamenti

Examples of GAN samples



32

In grado di prendere un input e riprodurre l'immagine come dipinti in certi stili, o viceversa.

Examples of GAN samples



33

Posso usare le GAN per fare anche design, faccio forme e metto vincoli fisici.

Stesse caratteristiche di torsione e forza, ma con meno materiale.

Il discriminatore è supervised, il generatore no.

Deep Learning - lecture 9

Reinforcement learning

Prof. Stefano Carrazza

University of Milan and INFN Milan

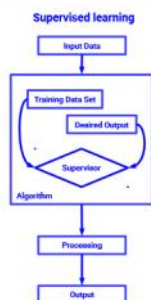
Reinforcement learning

Questa tecnica permette di imparare tramite un'interazione diretta con l'ambiente (un ambiente simulato). Può essere utile per ottimizzare in ambienti complessi, o nel controllare dinamicamente, o trovare buoni parametri? Se esiste spazio stato/azione, una struttura reinforcement learning può essere utile.

Supervised vs unsupervised learning

Supervised Learning:

- **Data** and **labels** are available.
- **Examples:** regression, classification, localization, object detection, segmentation, etc.

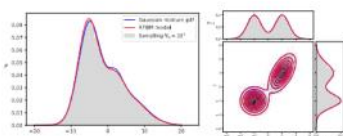


Recap: nel supervised learning ho questo. L'algoritmo impara a detectare gli oggetti e labellarli

Supervised vs unsupervised learning

Unsupervised Learning:

- **Data** and **no labels!**.
- **Examples:** density estimation, clustering, dimensionality reduction, etc.



Unsupervised learning

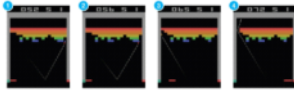


Poi abbiamo parlato di modelli generativi, con tecniche non supervisionati. Abbiamo quindi dati senza etichette.

Reinforcement Learning

Reinforcement Learning:

- **Agent** interacting with an **environment**, which provides numeric reward signals.
- Goal: Learn how to take optimal actions to maximize reward.



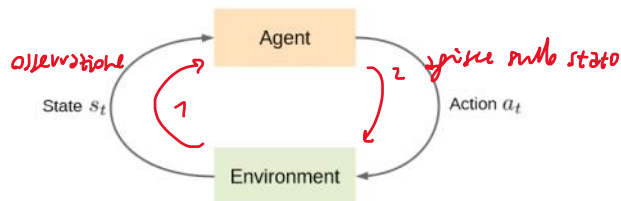
3

Qui non abbiamo etichette, ma un agente che interagisce con l'ambiente, che restituisce valori come reward. Questo concetto di reward, è la ricompensa che il modello ottiene dall'interazione con l'ambiente. L'obiettivo dell'agente è come agire in modo da aumentare la cumulativa dei reward (accumulare il massimo reward).

Il videogioco è un buon setup perché è un environment controllato, il reward è facile, e le azioni sono limitate.

What is Reinforcement Learning?

Reinforcement Learning



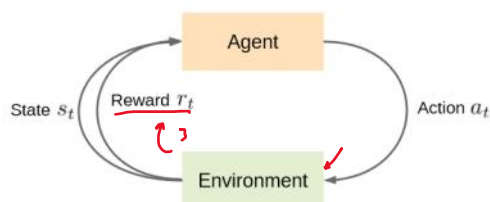
4

La policy dell'agente sceglie le azioni, a partire dallo stato.

L'agente quindi:

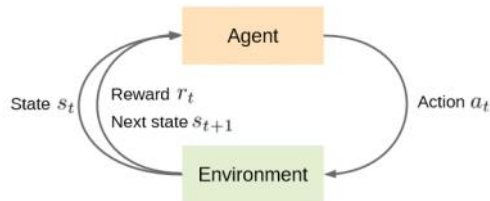
- Osserva lo stato
- La policy seleziona un'azione
- Il mondo fornisce un valore numerico che stima il valore dell'azione

Reinforcement Learning



5

Reinforcement Learning



6

E poi si itera

A differenza del supervised learning, in questo caso la ricompensa potrebbe arrivare più tardi dell'azione. Esiste una sorta di correlazione/effetto a catena tra le azioni. Per esempio una mossa sbagliata può portare ad una sconfitta dopo molte mosse. Questo rende molto complicato il problema rispetto i modelli supervisionati.

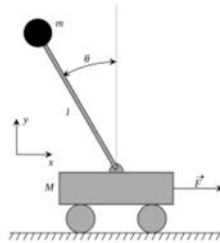
Cart-Pole example

Goal: Keep the pole upright on top of a movable cart.

State: angle, angular speed, position, horizontal velocity.

Action: horizontal force applied on the cart.

Reward: +1 at each time step if the pole is upright.



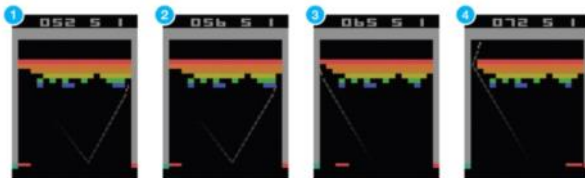
7

Questo esempio, consiste nel bilanciare una pallina.

La differenza tra reinforcement è che questo tiene conto della sequenza temporale, e non tanto ad ogni istante risolvo il problema, senza poi sapere se a tempi successivi la mia correzione dà problemi.

Non basta correggere, perché magari correzioni diventano sempre più forti, fin quando non sono più applicabili

Game example



Goal: Complete the game with highest score.

State: angle, angular speed, position, horizontal velocity.

Action: game controls (keyboard).

Reward: 1 at each time step if the pole is upright.

8

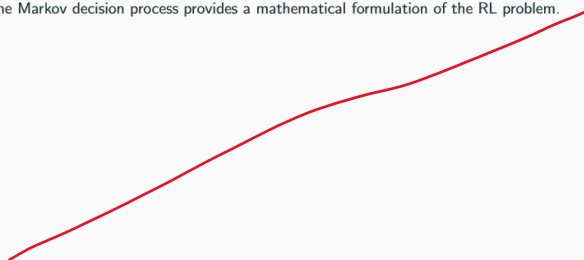
Un secondo esempio è vincere a questo atari game.

In questo caso la dimensionalità del problema è molto alta, ho tutte le varie informazioni. E qua devo poi trovare varie strategie. Riesce quindi a imparare anche cose complesse

Markov Decision Processes

Markov Decision Process

The Markov decision process provides a mathematical formulation of the RL problem.



9

Markov Decision Process

The Markov decision process provides a mathematical formulation of the RL problem.

Markov property: current state characterises the state of the world:

$$\text{World} = (S, A, R, P, \gamma)$$

where

- S : set of possible states.
- A : set of possible actions.
- R : distribution of reward given (state, action).
- P : transition probability, distribution of next state given (state, action) pair.
- γ : discount factor.

9

Per formalizzare questi problemi guardiamo i Markov decision problem, lo stato corrente deve descrivere il mondo, e in linea di principio ci permette di avere una mappa del passato e futuro?

S : insieme dei possibili stati

A : insieme delle possibili azioni

R : reward in funzione di (stato, azione)

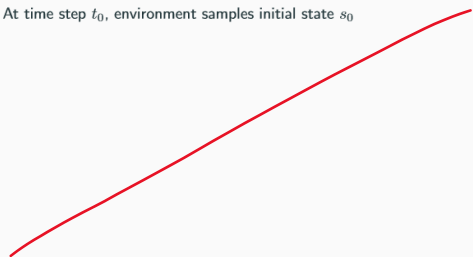
P : probabilità che dato (stato, azione) transisco allo stato s' , a seguito dell'azione scelta

Gamma: fattore di sconto, in $[0,1]$, importante delle ricompense future rispetto quelle immediate. Gamma circa 0 lo rende greedy, gamma circa 1 lo rende lungimirante.

Markov Decision Process

MDP algorithm:

- At time step t_0 , environment samples initial state s_0

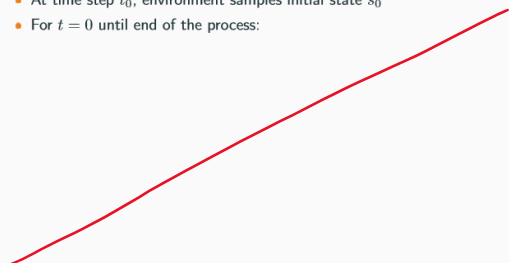


10

Markov Decision Process

MDP algorithm:

- At time step t_0 , environment samples initial state s_0
- For $t = 0$ until end of the process:



10

Markov Decision Process

MDP algorithm:

- At time step t_0 , environment samples initial state s_0
- For $t = 0$ until end of the process:
 - Agent selects action a_t (*Q: how?*)

10

Markov Decision Process

MDP algorithm:

- At time step t_0 , environment samples initial state s_0
- For $t = 0$ until end of the process:
 - Agent selects action a_t (*Q: how?*)
 - Environment samples reward $r_t \sim R(\cdot | s_t, a_t)$

10

Markov Decision Process

MDP algorithm:

- At time step t_0 , environment samples initial state s_0
- For $t = 0$ until end of the process:
 - Agent selects action a_t (*Q: how?*)
 - Environment samples reward $r_t \sim R(\cdot | s_t, a_t)$
 - Environment samples next state $s_{t+1} \sim P(\cdot | s_t, a_t)$

10

Markov Decision Process

MDP algorithm:

- At time step t_0 , environment samples initial state s_0
- For $t = 0$ until end of the process:
 - Agent selects action a_t (*Q: how?*)
 - Environment samples reward $r_t \sim R(\cdot | s_t, a_t)$
 - Environment samples next state $s_{t+1} \sim P(\cdot | s_t, a_t)$
 - Agent receives reward r_t and next state s_{t+1}

10

Markov Decision Process

MDP algorithm:

- At time step t_0 , environment samples initial state s_0
- For $t = 0$ until end of the process:
 - Agent selects action a_t (*Q: how?*)
 - Environment samples reward $r_t \sim R(\cdot | s_t, a_t)$
 - Environment samples next state $s_{t+1} \sim P(\cdot | s_t, a_t)$
 - Agent receives reward r_t and next state s_{t+1}

10

Markov Decision Process

MDP algorithm:

- At time step t_0 , environment samples initial state s_0
- For $t = 0$ until end of the process:
 - Agent selects action a_t (*Q: how?*)
 - Environment samples reward $r_t \sim R(\cdot | s_t, a_t)$
 - Environment samples next state $s_{t+1} \sim P(\cdot | s_t, a_t)$
 - Agent receives reward r_t and next state s_{t+1}

A policy π is a function from S to A that specifies what action to take in each state.

Goal: find a policy π^* that maximizes cumulative discounted reward:

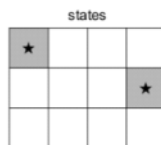
$$\sum_{t \geq 0} \gamma^t r_t$$

10

Optimal policy

Example of MDP: Grid World

actions = {
1. right →
2. left ←
3. up ↑
4. down ↓
}



Set a negative "reward"
for each transition
(e.g. $r = -1$)

Goal: reach one of greyed cells in least number of actions.

11

l'MDP funziona:

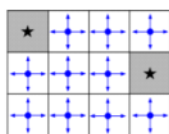
t=0 ho uno stato iniziale, per ogni passo poi:

- Agente seleziona un'azione (domanda chiave, come?)
- l'ambiente dà un reward
- l'ambiente finisce con una qualche probabilità lo stato successivo
- l'agente aggiorna il suo stato, e ripete

La policy π , è una funzione che mappa una funzione ad uno stato, quindi dato lo stato è quella che seleziona cosa fare. Posso poi considerarla deterministica(max) o statistica(sample sulle azioni).

La soluzione è la π che massimizza la cumulative delle ricompense

Example of MDP: Grid World



Random Policy

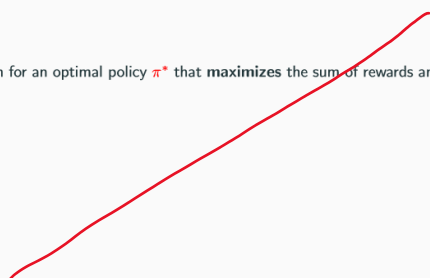


Optimal Policy

12

Optimal policy

Search for an optimal policy π^* that maximizes the sum of rewards and handles randomness.



13

Optimal policy

Search for an optimal policy π^* that maximizes the sum of rewards and handles randomness.

Formally:

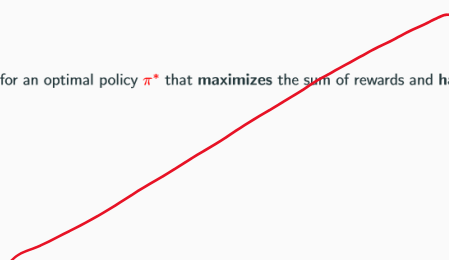
$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{t \geq 0} \gamma^t r_t | \pi \right],$$

with $s_0 \sim p(s_0)$, $a_t \sim \pi(\cdot | s_t)$, $s_{t+1} \sim p(\cdot | s_t, a_t)$

13

Optimal policy

Search for an optimal policy π^* that maximizes the sum of rewards and handles randomness.



14

Un esempio è una random policy, scelgo azioni a caso

O sennò la optimal ha un set di azioni ottimali (equivalenti) che posso seguire

Questo è la formulazione di quello che voglio

Optimal policy

Search for an optimal policy π^* that maximizes the sum of rewards and handles randomness.

Formally:

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{t \geq 0} \gamma^t r_t | \pi \right],$$

with $s_0 \sim p(s_0)$, $a_t \sim \pi(\cdot | s_t)$, $s_{t+1} \sim p(\cdot | s_t, a_t)$

14

Value function

Supposing to have a policy which produces samples $s_0, a_0, r_0, s_1, a_1, r_1, \dots$

We define the **value function** at state s following the policy π :

$$V^{\pi}(s) = \mathbb{E} \left[\sum_{t \geq 0} \gamma^t r_t | s_0 = s, \pi \right].$$

$V^{\pi}(s)$ measures how good is a state s .

15

Q-value function

Supposing to have a policy which produces samples $s_0, a_0, r_0, s_1, a_1, r_1, \dots$

Similarly, we define the **Q-value function** at state s and action a , following the policy π as

$$Q^{\pi}(s, a) = \mathbb{E} \left[\sum_{t \geq 0} \gamma^t r_t | s_0 = s, a_0 = a, \pi \right].$$

$Q^{\pi}(s, a)$ measures how good is a state-action pair.

16

Optimal state-action values

The optimal Q-value function, Q^* , is the maximum expected reward

$$Q^*(s, a) = \max_{\pi} \mathbb{E} \left[\sum_{t \geq 0} \gamma^t r_t | s_0 = s, a_0 = a, \pi \right]$$

Moreover, Q^* satisfies the following **Bellman equation**:

$$Q^*(s, a) = \mathbb{E}_{s' \sim \pi} \left[r + \gamma \max_{a'} Q^*(s', a') | s, a \right]$$

If the optimal state-action values for the next time-step $Q^*(s', a')$ are known, then the optimal strategy is to take the action that maximizes the expected value of $r + \gamma Q^*(s', a')$

The optimal policy π^* corresponds to taking the best action in any state as specified by Q^* .

17

Q-learning

The **value iteration algorithm** uses Bellman equation as an iterative update

$$Q_{i+1}(s, a) = \mathbb{E} \left[r + \gamma \max_{a'} Q_i(s', a') | s, a \right]$$

where Q_i converges to Q^* as $i \rightarrow \infty$

L'obiettivo è trovare π^* . Per farlo posso usare aiuti. Posso definire una funzione di valore (value function), che dice quanto è buono uno stato s , seguendo la policy π . Un modo per calcolarlo è considerare la somma delle ricompense scontate. Questo si chiama ritorno atteso

Avendo la value function, posso avere una strategia, la Q value function, che con delle modifiche posso arrivare alla policy π . Qua Q dipende quindi dallo spazio monodimensionale degli stati, a una funzione che misura il valore atteso:

- Partendo dallo stato s
- Facendo l'azione a ora
- E poi seguendo la policy (ottimale se Q^*) π

Vogliamo quindi l'optimum value di questa Q. Possiamo definire la Q^* , come agendo s , e poi seguendo la policy ottimale. Vogliamo quindi un modo per massimizzare questa quantità.

Possiamo poi dire che Q^* soddisfa la Bellman equation, dove la $Q^*(s, a)$ è

Questa formula dice quindi un modo di calcolare Q^* , in modo ricorsivo. Quindi se conosciamo Q^* , allora scelgo l' a' che massimizza $Q^*(s, a')$

Quindi non devo cercare un'infinita di soluzioni, ma uso una sorta di formula ricorsiva, per costruire la Q^* e quindi la policy per approssimazioni successive

18

Q-learning

The **value iteration algorithm** uses Bellman equation as an iterative update

$$Q_{i+1}(s, a) = \mathbb{E} \left[r + \gamma \max_{a'} Q_i(s', a') | s, a \right]$$

where Q_i converges to Q^* as $i \rightarrow \infty$

In order to avoid the computation of $Q(s, a)$ for every state-action pair (which in many cases is computationally infeasible) we use a **function approximator**:

$$Q(s, a; \theta) \approx Q^*(s, a)$$

where θ are the model parameters.

18

Q-learning

The **value iteration algorithm** uses Bellman equation as an iterative update

$$Q_{i+1}(s, a) = \mathbb{E} \left[r + \gamma \max_{a'} Q_i(s', a') | s, a \right]$$

where Q_i converges to Q^* as $i \rightarrow \infty$

In order to avoid the computation of $Q(s, a)$ for every state-action pair (which in many cases is computationally infeasible) we use a **function approximator**:

$$Q(s, a; \theta) \approx Q^*(s, a)$$

where θ are the model parameters.

If the model is a deep neural network $\rightarrow$ **Deep Q-Learning** (Mnih et al, arXiv:1312.5602)

18

Questo è un modo di fare Qlearning. Vogliamo ora una formulazione con deep learning che ci permetta di ottenere la policy migliore.

Per la maggior parte dei problemi, il numero di stati/azioni è così grande che è infattibile. Se si lavora in spazi troppo grandi c'è bisogno di un altro modo.

In modo analogo all'autoencoder, posso definire una $Q(s, a | \theta)$ circa $Q^*(s, a)$. Quindi se il modello che approssima Q^* è una rete neurale, la chiamo Deep Q-Learning.

Q-learning

Find a Q-function that satisfies the Bellman equation:

$$L_i(\theta_i) = \mathbb{E}_{s, a \sim \rho(\cdot)} \left[(y_i - Q(s, a; \theta_i))^2 \right]$$

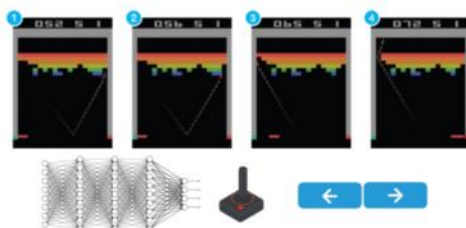
where

$$y_i = \mathbb{E}_{s' \sim \epsilon} \left[r + \gamma \max_{a'} Q(s', a'; \theta_{i-1}) | s, a \right]$$

19

Example

Playing ATARI games with DRL from Minh et al., arXiv:1312.5602, Nature'15

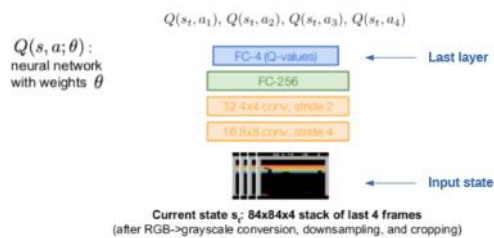


20

L'obiettivo è quindi di trovare una Q function che soddisfi l'equazione di Bellmann.

La cosa più facile è minimizzare una funzione di perdita L_i , che è una mse, dove la y_i è il target, che calcolo con l'equazione di Bellmann, e dipende da θ_{i-1} , ossia dallo step precedente, e questo stabilizza l'apprendimento perché ho un'evoluzione temporale.

Q-learning



21

Experience Replay

Algorithm 1 Deep Q-learning with Experience Replay

Initialize replay memory $\mathcal{D}$ to capacity N

Initialize action-value function Q with random weights

for episode = 1, M do

 Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequence $\phi_1 = \phi(s_1)$

 for $t = 1, T$ do

 With probability ϵ select a random action a_t

 otherwise select $a_t = \arg \max_a Q(\phi(s_t), a; \theta)$

 Execute action a_t in emulator and observe reward r_t and image x_{t+1}

 Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$

 Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in $\mathcal{D}$

 Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from $\mathcal{D}$

 Set $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$

 Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ according to equation 3

 end for

end for

22

Experience Replay

Algorithm 1 Deep Q-learning with Experience Replay

Initialize replay memory $\mathcal{D}$ to capacity N ← Initialize replay memory, Q-network

Initialize action-value function Q with random weights ← Play M episodes

for episode = 1, M do

 Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequence $\phi_1 = \phi(s_1)$

 for $t = 1, T$ do

 With probability ϵ select a random action a_t

 otherwise select $a_t = \arg \max_a Q(\phi(s_t), a; \theta)$

 Execute action a_t in emulator and observe reward r_t and image x_{t+1}

 Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$

 Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in $\mathcal{D}$ ← Store transition in replay memory

 Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from $\mathcal{D}$ ← Experience Replay

 Set $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$

 Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ according to equation 3

 end for

end for

23

Q-learning

<https://www.youtube.com/watch?v=V1eYniJ0Rnk>

24

Una sottigliezza è che l'algoritmo è un po' più complesso

L'altra innovazione è l'experience replay. Quando l'agente interagisce con l'ambiente, le cose vengono memorizzate in un memory buffer. In questo modo al posto di usare solo l'ultima transizione, usano una memoria delle ultime transizioni fatte in modo da accelerare l'apprendimento.

Vantaggi:

- Le transizioni successive sono correlate, in questo modo rompo le correlazioni.
- Riutilizzo i dati, quindi più efficiente
- Ho poi anche apprendimento offpolicy, ossia imparo in modo diverso, questo mi permette una sorta di adattamento da un altro testset, quindi una sorta di transfer learning

È un paradigma difficile da tenere sotto controllo. Vanno ridefinite le reward, e ritoccare varie cose. Ci sono poi problemi aperti, per esempio la dimensionalità. Come trovare le migliori azioni, con ricompense molto rare è anche una difficoltà. Come fare in modo che l'agente non faccia operazioni dannose, che per esempio disturbi la safety. E anche l'interpretazione del perché l'agente faccia delle scelte. Questo modello ha un'attitudine di real time application